# Outline

- Planning systems
- PDDL (Planning Domain Definition Language)
- Planning algorithms
  - Forward chaining
  - Backward chaining
  - Partial-order planning
- Applications

# Planning: Motivation

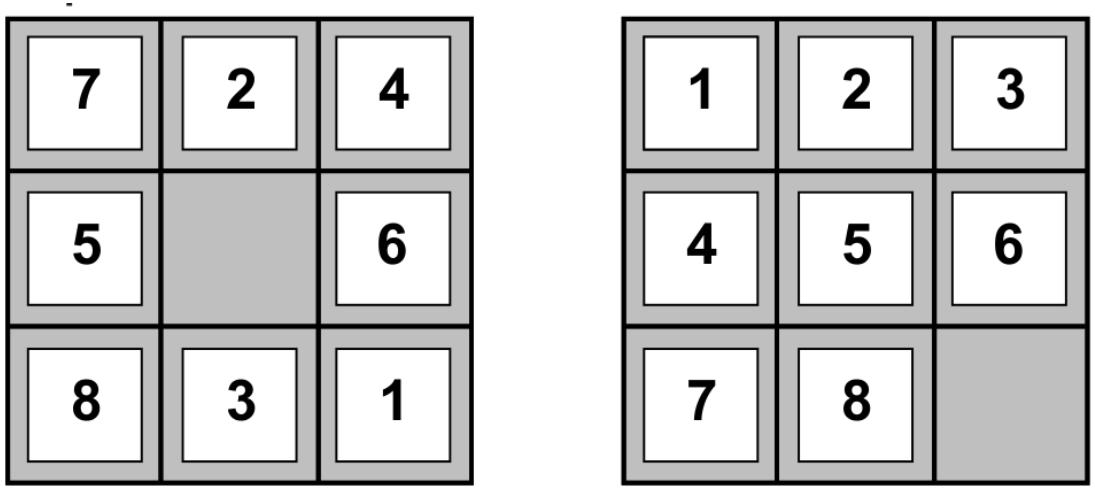
How to develop systems or 'agents'
that can make decisions on their own?

# What is AI planning?

*Planning* is the task of finding a set of actions that will achieve a goal. A *planner* is a program that searches for a plan. It inputs a description of the world and the goals. The output is a plan. The simplest plan is a sequence of actions:

```
``do action1, do action2 ...''
```

More complex plans may include branching actions: “if (condition) do action1 else do action2”



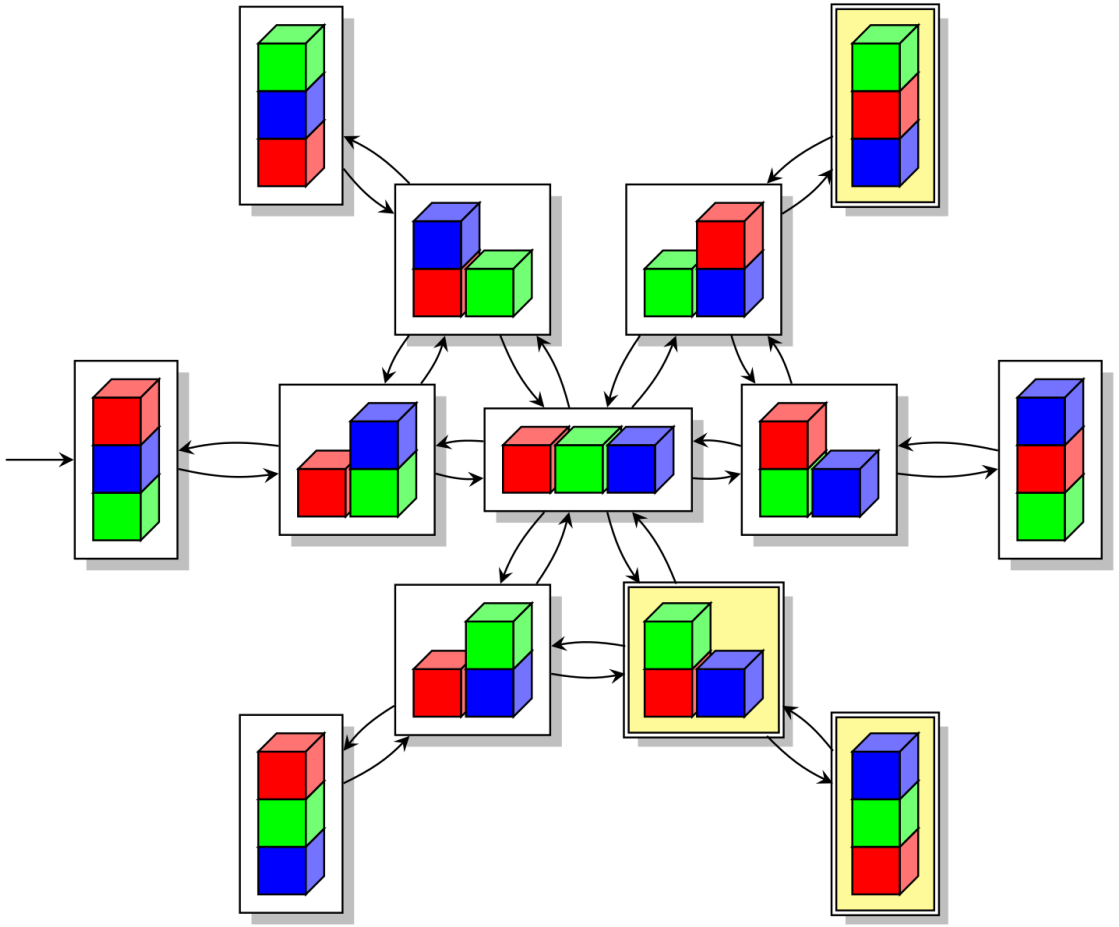

Start State  Goal State

## Definition (Plan)

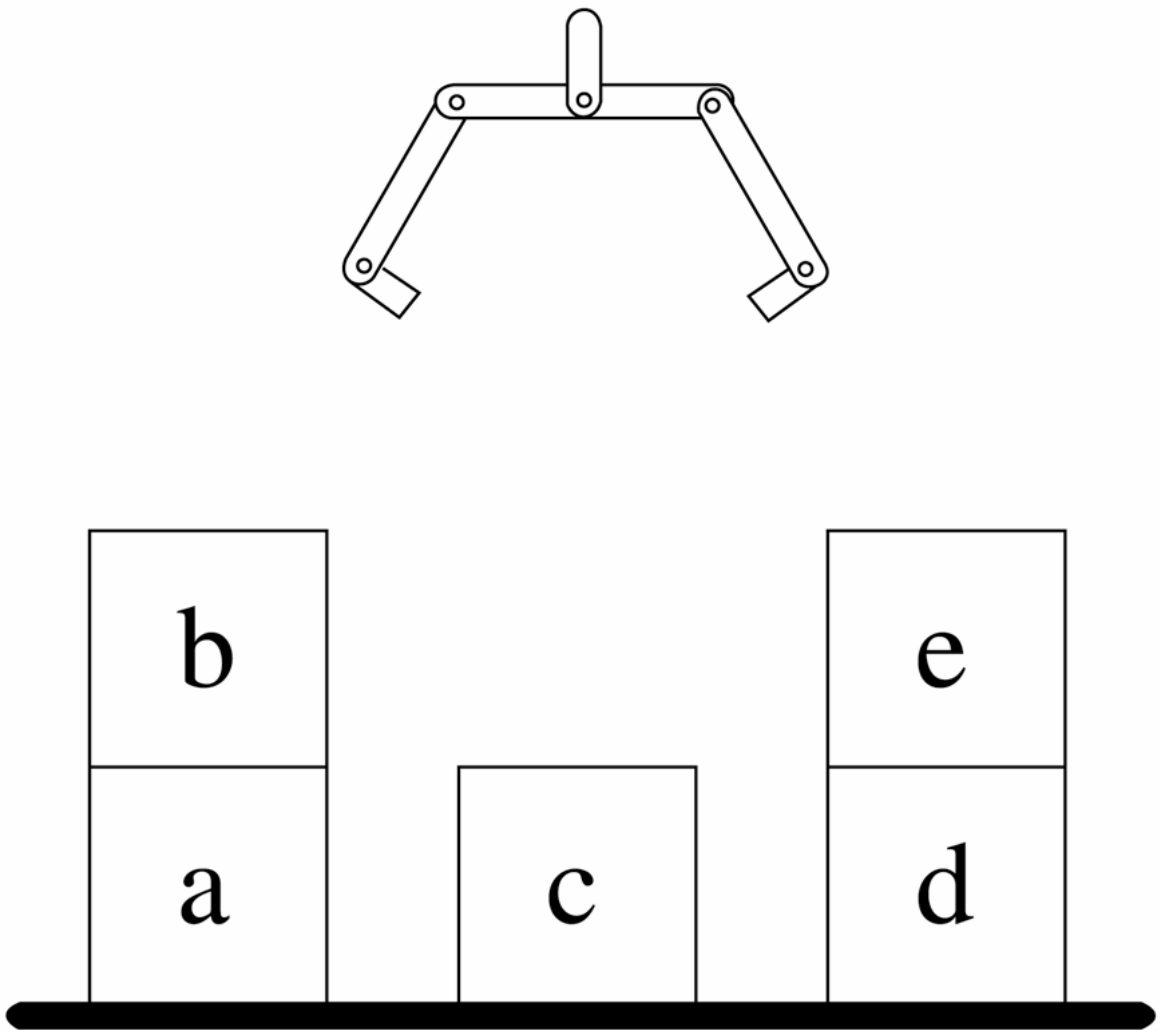
A plan for a transition system is a sequence of actions occurring as labels on a path from the initial state to a goal state.

The cost of a plan $\langle a_1, \ldots, a_n \rangle$ is $\sum_{i=1}^{n} cost(a_i)$.

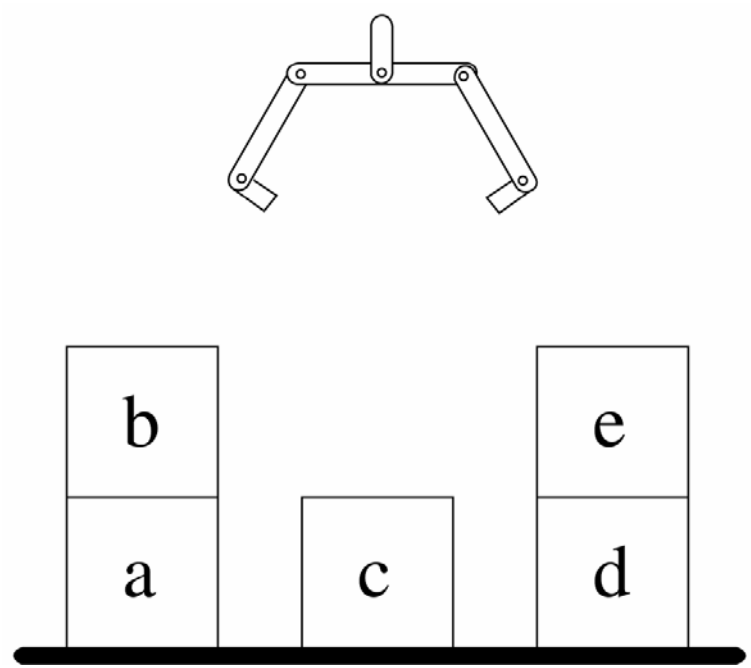
A plan is optimal if it has minimal cost.



# The blocks world



# Represent this world using predicates




**ontable(a)**
**ontable(c)**
**ontable(d)**
**on(b,a)**
**on(e,d)**
**clear(b)**
**clear(c)**
**clear(e)**
**gripping()**

# A state space algorithm for STRIPS operators

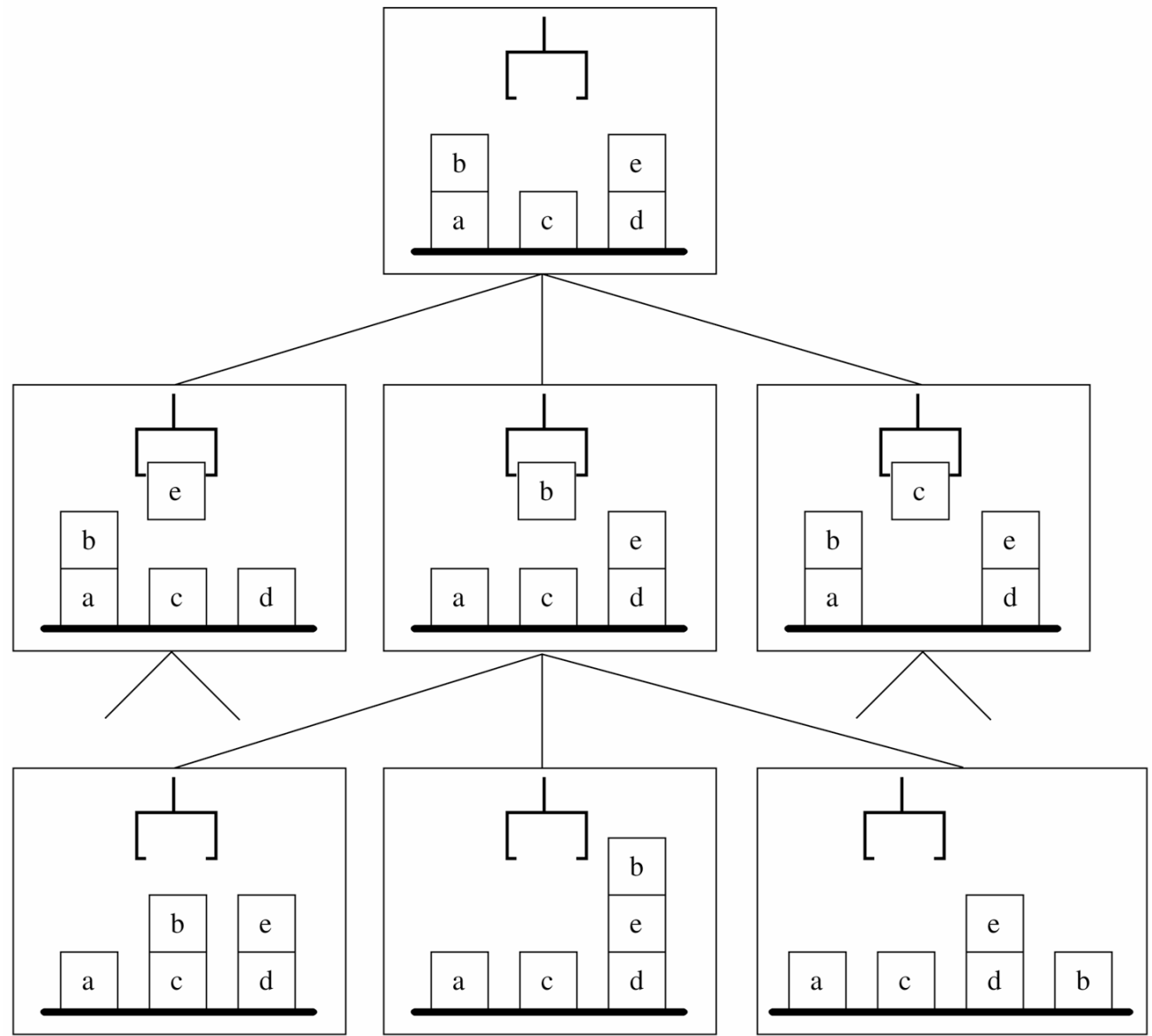
**Search the space of situations (or states). This means each node in the search tree is a state.**

**The root of the tree is the start state.**

**Operators are the means of transition from each node to its children.**

**The goal test involves seeing if the set of goals is a subset of the current situation.**

# Now, the following graph makes much more sense



# Autonomous Behavior in AI

The key problem is to select **the action to do next**. This is the so-called **control problem**. Three approaches to this problem:

- **Programming-based:** Specify control by hand
- **Learning-based:** Learn control from experience
- **Model-based:** Specify problem by hand, derive control automatically

**Planning** is the **model-based approach to autonomous behavior** where agent controller derived from model of the actions, sensors, and goals.

# Basic State Model: Classical Planning

- finite and discrete state space $S$
- a **known initial state** $s_0 \in S$
- a set $S_G \subseteq S$ of goal states
- actions $A(s) \subseteq A$ applicable in each $s \in S$
- a **deterministic transition function** $s' = f(a, s)$ for $a \in A(s)$
- positive **action costs** $c(a, s)$

A **solution** is a sequence of applicable actions that maps $s_0$ into $S_G$, and it is **optimal** if it minimizes **sum of action costs** (e.g., # of steps)

# A Basic Language for Classical Planning: Strips

- A **problem** in Strips is a tuple $P = \langle F, O, I, G \rangle$:
  - $F$ stands for set of all **atoms** (boolean vars)
  - $O$ stands for set of all **operators** (actions)
  - $I \subseteq F$ stands for **initial situation**
  - $G \subseteq F$ stands for **goal situation**

- Operators $o \in O$ **represented** by
  - the **Add** list $Add(o) \subseteq F$
  - the **Delete** list $Del(o) \subseteq F$
  - the **Precondition** list $Pre(o) \subseteq F$

# From Language to Models

A Strips problem $P = \langle F, O, I, G \rangle$ determines **state model** $\mathcal{S}(P)$ where

- the states $s \in S$ are **collections of atoms** from $F$
- the initial state $s_0$ is $I$
- the goal states $s$ are such that $G \subseteq s$
- the actions $a$ in $A(s)$ are ops in $O$ s.t. $Prec(a) \subseteq s$
- the next state is $s' = s - Del(a) + Add(a)$
- action costs $c(a, s)$ are all $1$

– (Optimal) **Solution** of $P$ is (optimal) **solution** of $\mathcal{S}(P)$

– Slight language extensions often convenient: **negation**, **conditional effects**, **non-boolean variables**; some required for describing richer models (costs, probabilities, ...).

# Characteristics of *classical planners*

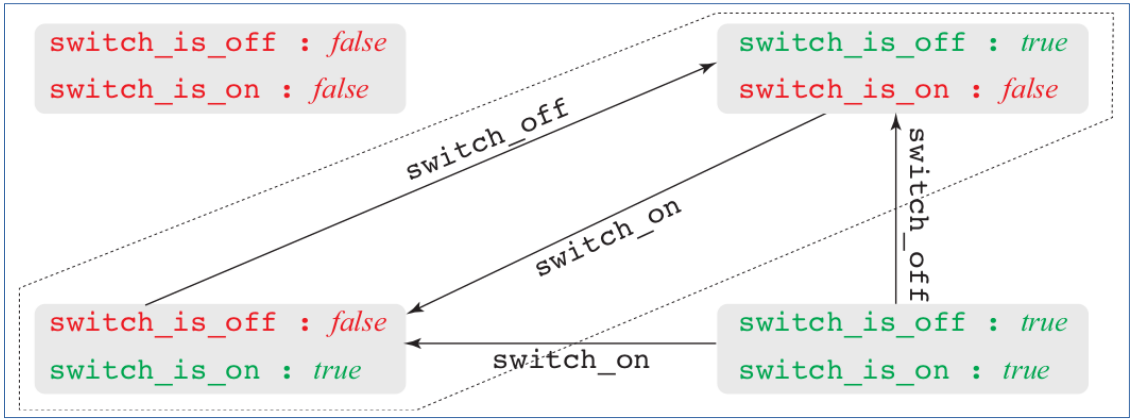
- **They operate on basic STRIPS actions.**
- **Important assumptions:**
  - **the agent is the only source of change in the world, otherwise the environment is static.**
  - **all the actions are deterministic.**
  - **the agent is omniscient: knows everything it needs to know about start state and effects of actions.**
  - **the goals are categorical, the plan is considered successful iff all the goals are achieved.**

# How to define a planning problem

- **Create a *domain file:* contains the domain behavior, simply the operators**
- **Create a *problem file:* contains the initial state and the goal**

## Domain.pddl

```
(define (domain switch)
    (:requirements :strips)
    (:predicates    (switch_is_on)
                    (switch_is_off)
    )
    (:action switch_on
        :precondition (switch_is_off)
        :effect (and    (switch_is_on)
                        (not (switch_is_off))
                )
    )
    (:action switch_off
        :precondition (switch_is_on)
        :effect (and    (switch_is_off)
                        (not (switch_is_on))
                )
    )
)
```




## Problem.pddl

```
(define (problem turn_it_off)
    (:domain switch)
    (:init
        (switch_is_on)
    )
    (:goal
        (switch_is_off)
    )
)
```

## Plan

(switch_off)

# The robot arm can perform these tasks

- **pickup (W)**: pick up block W from its current location on the table and hold it
- **putdown (W)**: place block W on the table
- **stack (U, V)**: place block U on top of block V
- **unstack (U, V)**: remove block U from the top of block V and hold it

All assume that the robot arm can precisely reach the block.

## Domain.pddl

```
(define (domain BLOCKS)
  (:requirements :strips)
  (:predicates (on ?x ?y)
               (ontable ?x)
               (clear ?x)
               (handempty)
               (holding ?x)
               )

  (:action pick-up
           :parameters (?x)
           :precondition (and (clear ?x)
              (ontable ?x) (handempty))
           :effect
           (and (not (ontable ?x))
             (not (clear ?x))
             (not (handempty))
             (holding ?x)))

  (:action put-down
           :parameters (?x)
           :precondition (holding ?x)
           :effect
           (and (not (holding ?x))
             (clear ?x)
             (handempty)
             (ontable ?x)))

  (:action stack
           :parameters (?x ?y)
           :precondition (and (holding ?x)
              (clear ?y))
           :effect
           (and (not (holding ?x))
             (not (clear ?y))
             (clear ?x)
             (handempty)
             (on ?x ?y)))
  (:action unstack
           :parameters (?x ?y)
           :precondition (and (on ?x ?y)
              (clear ?x) (handempty))
           :effect
           (and (holding ?x)
             (clear ?y)
             (not (clear ?x))
             (not (handempty))
             (not (on ?x ?y)))))
```

## Problem.pddl

```pddl
(define (problem BLOCKS-5-0)
    (:domain BLOCKS)
    (:objects B E A C D )
    (:init
        (CLEAR D)
        (CLEAR C)
        (ONTABLE D)
        (ONTABLE A)
        (ON C E)
        (ON E B)
        (ON B A)
        (HANDEMPTY)
    )
    (:goal
        (AND
            (ON A E)
            (ON E B)
            (ON B D)
            (ON D C)
        )
    )
)
```

## Plan >>

- (unstack c e)
- (put-down c)
- (pick-up d)
- (stack d c)
- (unstack e b)
- (put-down e)
- (unstack b a)
- (stack b d)
- (pick-up e)
- (stack e b)
- (pick-up a)
- (stack a e)

```pddl
(:action unstack
  :parameters (c e)
  :precondition
    (and
      (on c e)
      (clear c)
      (handempty)
    )
  :effect
    (and
      (holding c)
      (clear e)
      (not
        (clear c)
      )
      (not
        (handempty)
      )
      (not
        (on c e)
      )
    )
)
```

**Domain.pddl (3 operators)**

```
(define (domain blocks_world)
  (:requirements :strips)
  (:predicates (on-table ?x) (on ?x ?y) (clear ?x))

  (:action Move_from_Stack_to_Table
    :parameters (?obj ?src)
    :precondition (and (clear ?obj) (on ?obj ?src))
    :effect (and (clear ?src) (on-table ?obj) (not (on ?obj ?src))))

  (:action Move_from_Stack_to_Stack
    :parameters (?obj ?src ?dst)
    :precondition (and (clear ?obj) (clear ?dst) (on ?obj ?src))
    :effect (and (clear ?src) (on ?obj ?dst) (not (clear ?dst)) (not (on ?obj ?src))))

  (:action Move_from_Table_to_Stack
    :parameters (?obj ?dst)
    :precondition (and (clear ?obj) (clear ?dst) (on-table ?obj))
    :effect (and (on ?obj ?dst) (not (clear ?dst)) (not (on-table ?obj))))
)
```

**Problem.pddl**

```
(define (problem bw-12step)
    (:domain blocks_world)
    (:objects A B C D E F G)
    (:init  (on-table A) (clear A) (on-table B) (clear B) (on-table G) (on F G)
            (on E F) (on D E) (on C D) (clear C))
    (:goal (and (on B C) (on-table A) (on F A) (on C D)))
)
```

**Domain.pddl (2 operators)**

```
(define (domain simple-blocks)
  (:requirements :strips :equality)
  (:predicates (on ?x ?y)
               (clear ?x))
  (:constants Table)

  (:action Move_to_table
           :parameters (?obj ?src)
           :precondition (and (on ?obj ?src) (clear ?obj)
                              (not (= ?obj Table)) (not (= ?src Table)))
           :effect (and (on ?obj Table) (clear ?src) (not (on ?obj ?src))))
  (:action Move_to_Stack
           :parameters (?obj ?src ?dst)
           :precondition (and (on ?obj ?src) (clear ?obj) (clear ?dst)
                              (not (= ?obj Table)) (not (= ?dst Table)) (not (= ?obj ?dst)))
           :effect (and (on ?obj ?dst) (clear ?src)
                        (not (on ?obj ?src)) (not (clear ?dst)))))
```

**Problem.pddl**

```
(define (problem simple-block1)
    (:domain simple-blocks)
  (:objects A B C)
  (:goal (and (on a b) (on b c)))
  (:init (on a table) (on c a) (clear c) (on b table) (clear b)))
```

**Domain.pddl (1 operator)**

```
(define (domain blocks-world-domain)
  (:requirements :strips :equality :conditional-effects)
  (:constants Table)
  (:predicates (on ?x ?y)
               (clear ?x)
               (block ?b)
               )

  (:action move
          :parameters (?obj ?src ?dst)
          :precondition (and (on ?obj ?src) (clear ?obj) (clear ?dst)
                  (not (= ?dst ?src)) (not (= ?obj ?src))
                  (not (= ?obj ?dst)) (not (= ?obj Table)))
          :effect
          (and (on ?obj ?dst) (not (on ?obj ?src))
            (when (not (= ?src Table)) (clear ?src))
            (when (not (= ?dst Table)) (not (clear ?dst))))))
```

**Problem.pddl**

```
(define (problem sussman-anomaly)
    (:domain blocks-world-domain)
  (:objects A B C)
  (:init (block A) (block B) (block C) (block Table)
     (on C A) (on A Table) (on B Table) (clear C) (clear B) (clear Table))
  (:goal (and (on B C) (on A B))))
```

**Domain.pddl (1 operator)**

```
(define (domain mcd-blocksworld)
  (:requirements :adl :universal-preconditions :disjunctive-preconditions)
  (:constants Table)
  (:predicates (on ?x ?y)
               (clear ?x)
               (block ?b)
               (above ?x ?y))
  (:action puton
           :parameters (?obj ?dst ?src)
           :precondition (and (not (= ?obj ?dst)) (not (= ?obj table)) (not (= ?src ?dst))
                   (on ?obj ?src)
                   (or (= ?obj Table)
                        (forall (?b) (imply (block ?b) (not (on ?b ?obj)))))
                   (or (= ?dst Table)
                        (forall (?b) (imply (block ?b) (not (on ?b ?dst))))))
           :effect
           (and (on ?obj ?dst) (not (on ?obj ?src))
             (forall (?c)
                  (when (or (= ?dst ?c) (above ?dst ?c))
                     (above ?obj ?c)))
             (forall (?e)
                  (when (and (above ?obj ?e) (not (= ?dst ?e))
                       (not (above ?dst ?e)))
                     (not (above ?obj ?e)))))))
```

**Problem.pddl**

```
(define (problem mcd-sussman-anomaly)
    (:domain mcd-blocksworld)
  (:objects A B C)
  (:init (block A) (block B) (block C) (block Table)
     (on c a) (on b table) (on a table))
  (:goal (and (on b c) (on a b))))
```

```
(define (domain knights-tour)
  (:requirements :negative-preconditions)

  (:predicates
    (at ?col ?row)
    (visited ?col ?row)
    (diff_by_one ?x ?y)
    (diff_by_two ?x ?y)
   )

  (:action move_2col_1row
   :parameters (?from_col ?from_row ?to_col ?to_row)
   :precondition (and (at ?from_col ?from_row)
                      (diff_by_two ?from_col ?to_col) ; col +/- 2
                      (diff_by_one ?from_row ?to_row) ; row +/- 1
                      (not (visited ?to_col ?to_row)))
   :effect (and (not (at ?from_col ?from_row))
                (at ?to_col ?to_row)
                (visited ?to_col ?to_row))
   )

  (:action move_2row_1col
   :parameters (?from_col ?from_row ?to_col ?to_row)
   :precondition (and (at ?from_col ?from_row)
                      (diff_by_two ?from_row ?to_row) ; row +/- 2
                      (diff_by_one ?from_col ?to_col) ; col +/- 1
                      (not (visited ?to_col ?to_row)))
   :effect (and (not (at ?from_col ?from_row))
                (at ?to_col ?to_row)
                (visited ?to_col ?to_row))
   )
)
```

| | | | | |
|---|---|---|---|---|
| ♞ | 14 | 9 | 20 | 3 |
| 24 | 19 | 2 | 15 | 10 |
| 13 | 8 | 23 | 4 | 21 |
| 18 | 25 | 6 | 11 | 16 |
| 7 | 12 | 17 | 22 | 5 |

```
(define (problem knights-tour-problem-5x5)
  (:domain knights-tour)

  ; Define a set of "numbers" 1..8:
  (:objects n1 n2 n3 n4 n5)

  (:init
    ; Initial position of the Knight piece
    ; (upper left corner):
    (at n1 n5)
    (visited n1 n5)

    ; Here, we have to
    ; of the static pre
    (diff_by_one n1 n2)
    (diff_by_one n2 n1)
    (diff_by_one n2 n3)
    (diff_by_one n3 n2)
    (diff_by_one n3 n4)
    (diff_by_one n4 n3)
    (diff_by_one n4 n5)
    (diff_by_one n5 n4)

    (diff_by_two n1 n3)
    (diff_by_two n3 n1)
    (diff_by_two n2 n4)
    (diff_by_two n4 n2)
    (diff_by_two n3 n5)
    (diff_by_two n5 n3)
  )
```

```
  (:goal (and (visited n1 n1)
              (visited n1 n2)
              (visited n1 n3)
              (visited n1 n4)
              (visited n1 n5)
              (visited n2 n1)
              (visited n2 n2)
              (visited n2 n3)
              (visited n2 n4)
              (visited n2 n5)
              (visited n3 n1)
              (visited n3 n2)
              (visited n3 n3)
              (visited n3 n4)
              (visited n3 n5)
              (visited n4 n1)
              (visited n4 n2)
              (visited n4 n3)
              (visited n4 n4)
              (visited n4 n5)
              (visited n5 n1)
              (visited n5 n2)
              (visited n5 n3)
              (visited n5 n4)
              (visited n5 n5)))
)
```

# Problems in representation

***Frame problem*: List everything that does not change. It no more is a significant problem because what is not listed as changing (via the add and delete lists) is assumed to be not changing.**

***Qualification problem*: Can we list every precondition for an action? For instance, in order for PICKUP to work, the block should not be glued to the table, it should not be nailed to the table, …**

**It still is a problem. A partial solution is to prioritize preconditions, i.e., separate out the preconditions that are worth achieving.**

# **Problems in representation** (cont'd)

***Ramification problem*: Can we list every result of an action? For instance, if a block is picked up its shadow changes location, the weight on the table decreases, ...**

**It still is a problem. A partial solution is to code rules so that inferences can be made. For instance, allow rules to calculate where the shadow would be, given the positions of the light source and the object. When the position of the object changes, its shadow changes too.**

# Why is planning a hard problem?

**It is due to the large branching factor and the overwhelming number of possibilities.**

**There is usually no way to separate out the relevant operators. Take the previous example, and imagine that there are 100 balls, just two rooms, and two grippers. Again, the goal is to take 1 ball to the other room.**

**How many PICK operators are possible in the initial situation?**

```
pick
        :parameters (?obj ?room ?gripper)
```

**That is only one part of the branching factor, the robot could also move without picking up anything.**

# **Why is planning a hard problem?** (cont'd)

**Also, goal interactions is a major problem. In planning, goal-directed search seems to make much more sense, but unfortunately cannot address the exponential explosion. This time, the branching factor increases due to the many ways of resolving the interactions.**

**When subgoals are compatible, i.e., they do not interact, they are said to be *linear* (or *independent*, or *serializable*).**

**Life is easier for a planner when the subgoals are independent because then divide-and-conquer works.**

# How to deal with the exponential explosion?

**Use goal-directed algorithms**

**Use domain-independent heuristics**

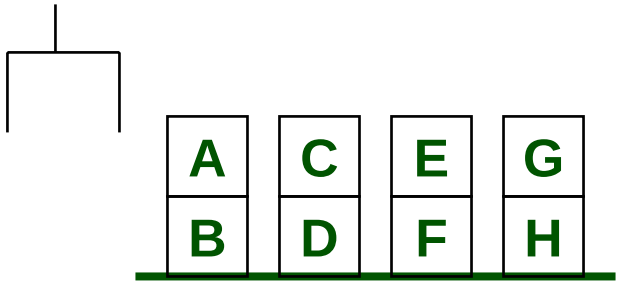
**Use domain-dependent heuristics**
**(we need a language to specify them)**

# A sampler of planning algorithms

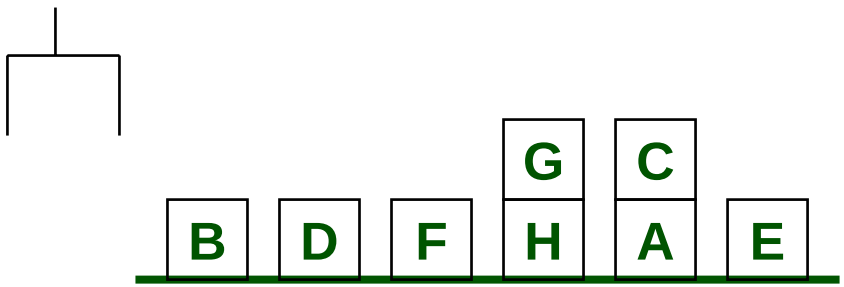
- **Forward chaining**
  - **Work in a state space**
  - **Start with the initial state, try to reach the goal state using forward progression**
- **Backward chaining**
  - **Work in a state space**
  - **Start with the goal state, try to reach the initial state using backward regression**
- **Partial order planning**
  - **Work in a plan space**
  - **Start with an empty plan, work from the goal to reach a complete plan**

# Forward chaining

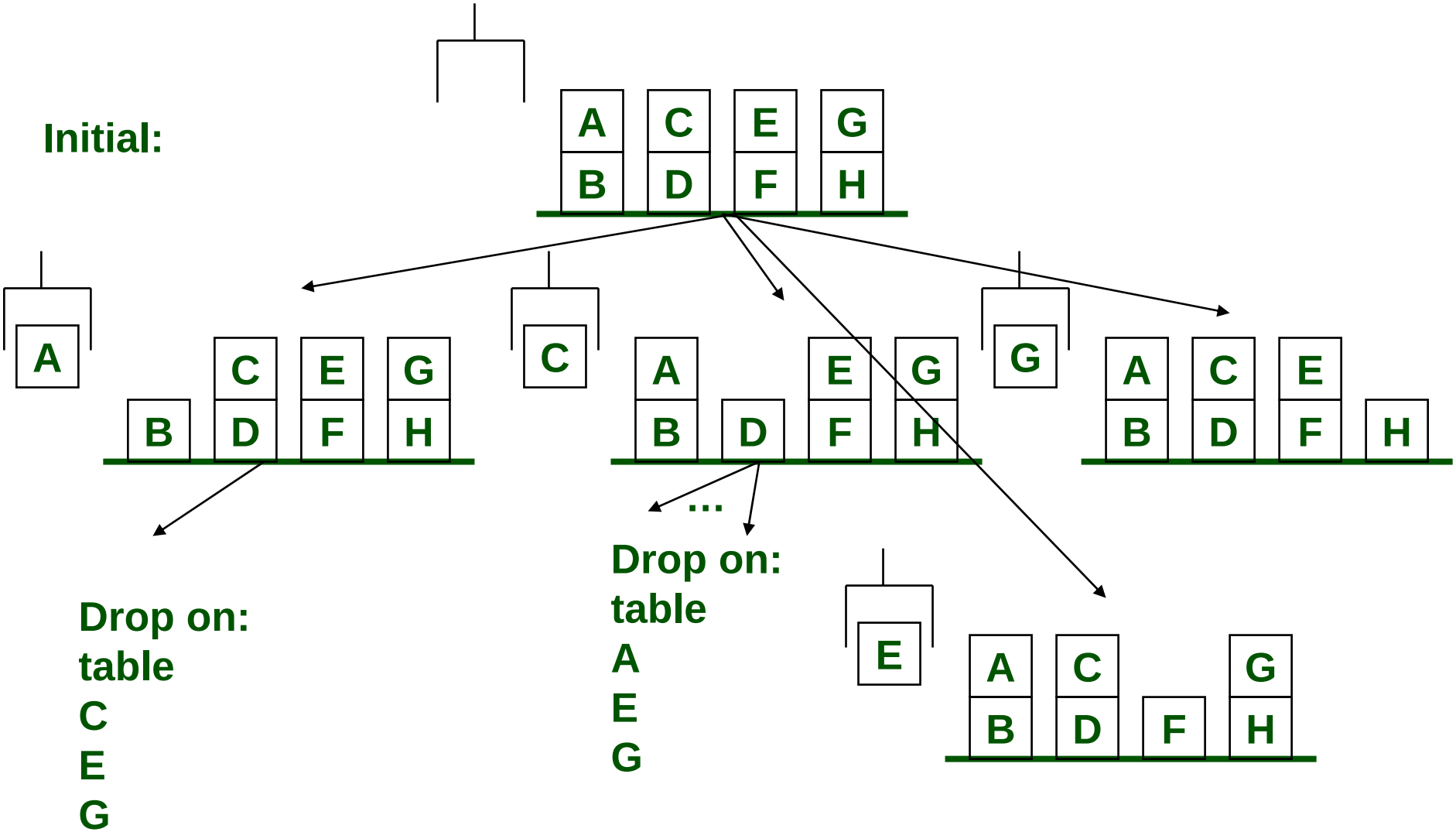
**Initial:**




**Goal :**



# 1st and 2nd levels of search

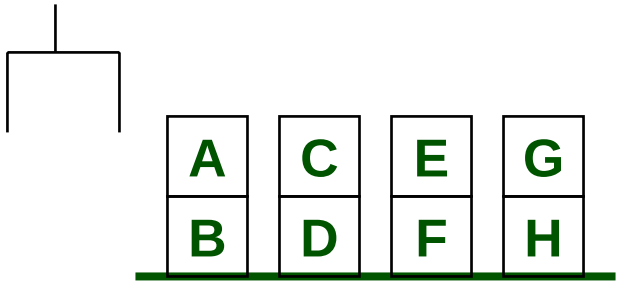


# Results

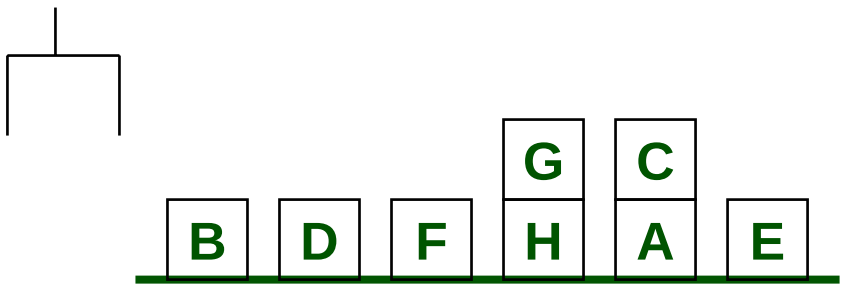
- **A plan is:**
  - **unstack (A, B)**
  - **putdown (A)**
  - **unstack (C, D)**
  - **stack (C, A)**
  - **unstack (E, F)**
  - **putdown (F)**
- **Notice that the final locations of D, F, G, and H need not be specified**
- **Also notice that D, F, G, and H will never need to be moved. But there are states in the search space which are a result of moving these. Working backwards from the goal might help.**

# Backward chaining

**Initial:**



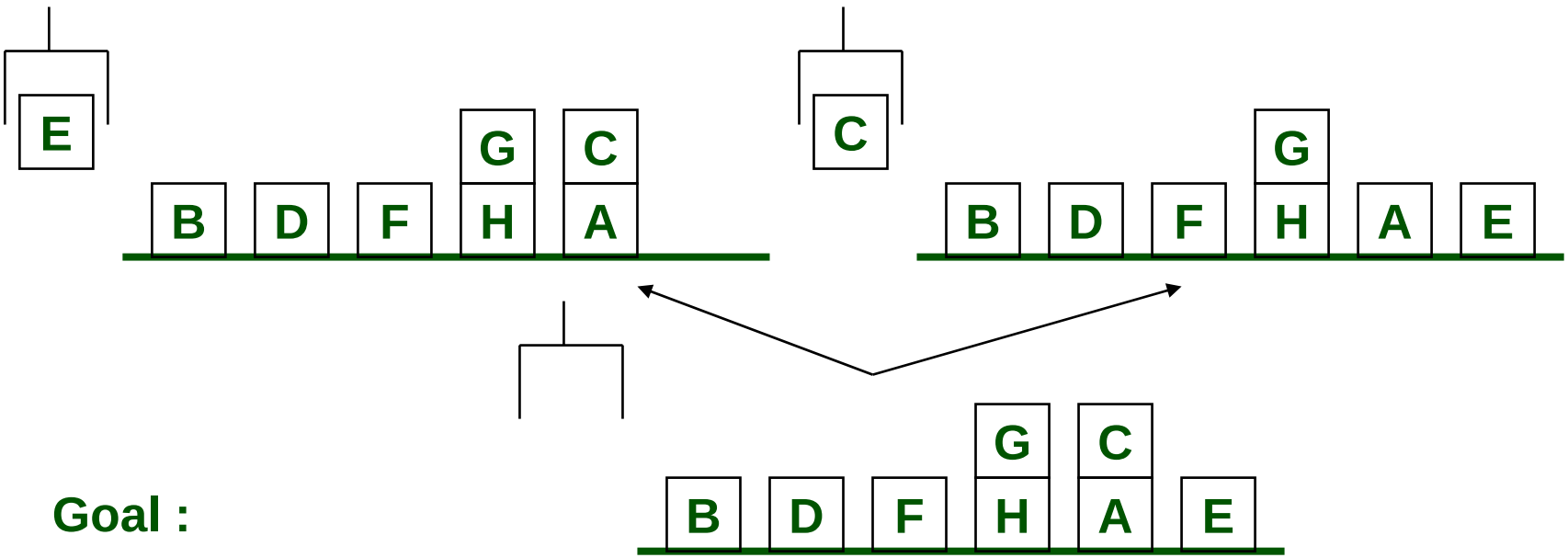

**Goal :**



# 1st level of search

**For E to be on the table, the last action must be putdown(E)**

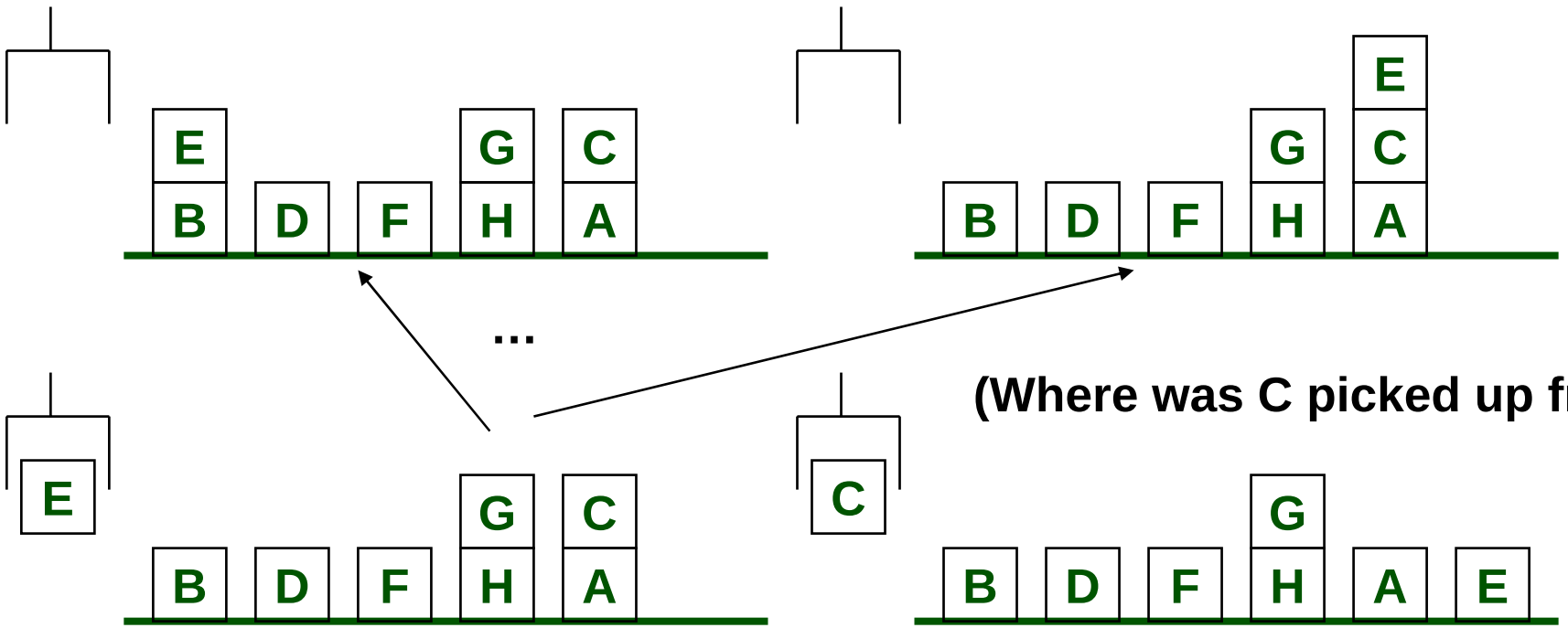
**For C to be on A, the last action must be stack(C,A)**




**Goal :**

# 2nd level of search

**Where was E picked up from?**




**(Where was C picked up from?)**

# Results

- **The same plan can be found**
  - **unstack (A, B)**
  - **putdown (A)**
  - **unstack (C, D)**
  - **stack (C, A)**
  - **unstack (E, F)**
  - **putdown (F)**
- **Now, the final locations of D, F, G, and H need to be specified**
- **Notice that D, F, G, and H will never need to be moved. But observe that from the second level on the branching factor is still high**

# Partial-order planning (POP)

- Notice that the resulting plan has two parallelizable threads:

| | | |
|---|---|---|
| unstack (A,B)<br>putdown (A)<br>unstack (C,D)<br>stack (C,A) | & | unstack (E, F)<br>putdown (F) |

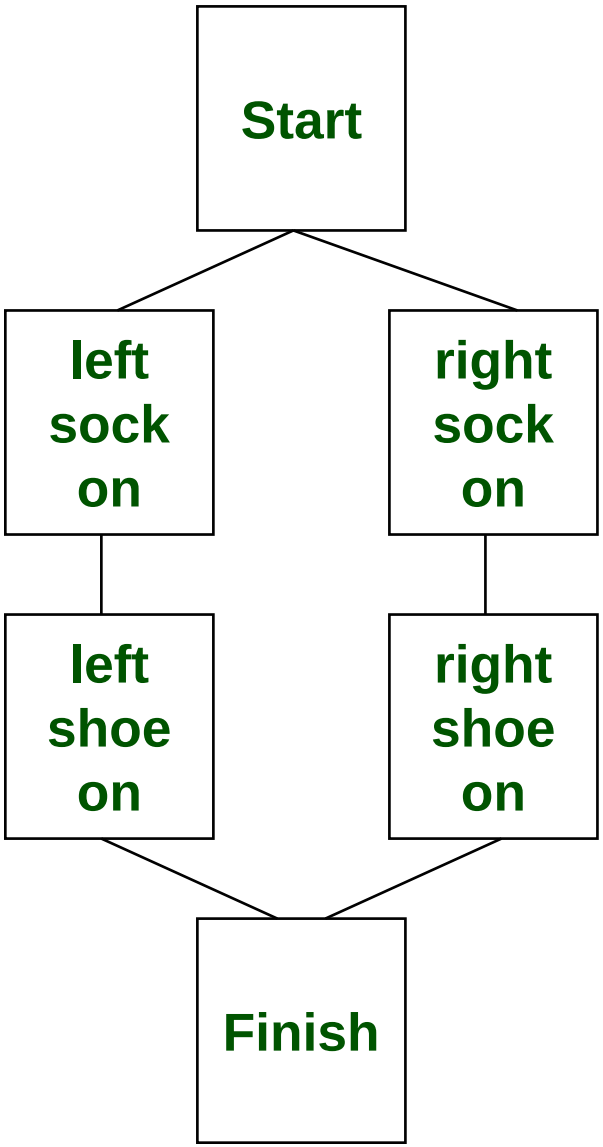
- These steps can be interleaved in 3 different ways:

| | | |
|---|---|---|
| unstack (E, F)<br>putdown (F)<br>unstack (A,B)<br>putdown (A)<br>unstack (C,D)<br>stack (C,A) | unstack (A,B)<br>putdown (A)<br>unstack (E, F)<br>putdown (F)<br>unstack (C,D)<br>stack (C,A) | unstack (A,B)<br>putdown (A)<br>unstack (C,D)<br>stack (C,A)<br>unstack (E, F)<br>putdown (F) |

# **Partial-order planning** (cont'd)

- **Idea: Do not order steps unless it is necessary**
- **Then a partially ordered plan represents several totally ordered plans**
- **That decreases the search space**
- **But still the planning problem is not solved, good heuristics are crucial**

# **Partial-order planning** (cont'd)




| | | | | | |
|---|---|---|---|---|---|
| Start | Start | Start | Start | Start | Start |
| Left sock on | Right sock on | Left sock on | Right sock on | Left sock on | Right sock on |
| Left shoe on | Right shoe on | Right sock on | Left sock on | Right sock on | Left sock on |
| Right sock on | Left sock on | Left shoe on | Right shoe on | Right shoe on | Left shoe on |
| Right shoe on | Left shoe on | Right shoe on | Left shoe on | Left shoe on | Right shoe on |
| Finish | Finish | Finish | Finish | Finish | Finish |

# Partially ordered plans

*Partially ordered* collection of steps with

- START step has the initial state description as its effect
- FINISH step has the goal description as its precondition
- causal links from outcome of one step to precondition of another
- temporal ordering between pairs of steps

# Partially ordered plans (cont'd)

A partially ordered plan is a 5-tuple (A, O, C, OC, UL)

- A is the set of actions that make up the plan. They are partially ordered.
- O is a set of ordering constraints of the form $A \prec B$. It means $A$ comes before $B$.
- C is the set of causal links in the form $(A, p, B)$ where $A$ is the *supplier action*, where $B$ is the *consumer action*, and $p$ is the condition supplied. It is read as "*A achieves p for B*."

# Partially ordered plans (cont'd)

A partially ordered plan is a 5-tuple (A, O, C, OC, UL)

- OC is a set of open conditions, i.e., conditions that are not yet supported by causal links. It is of the form $p$ for $A$ where $p$ is a condition and $A$ is an action.
- UL is a set of unsafe links, i.e., causal links whose conditions might be undone by other actions.

# Partially ordered plans (cont'd)

A plan is *complete* iff every precondition is achieved, and there are no unsafe links. A precondition is *achieved* iff it is the effect of an earlier step and no *possibly intervening* step undoes it

In other words, a plan is complete when $OC \cup UL = \emptyset$.

$OC \cup UL$ is referred to as the *flaws* in a plan.

When a causal link is established, the corresponding condition is said to be *closed*.

# Example

START

CleanLeftSock

CleanRightSock

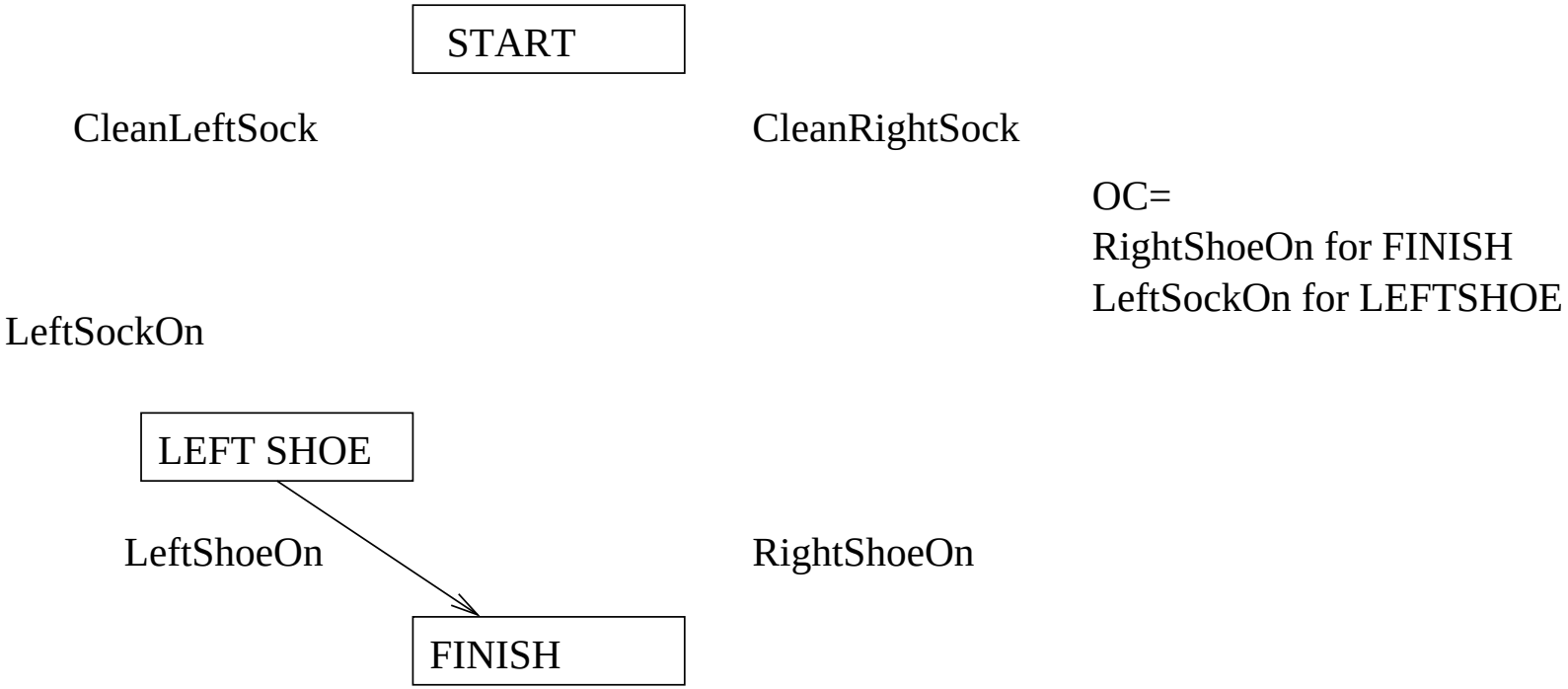
OC=
LeftShoeOn for FINISH
RightShoeOn for FINISH

LeftShoeOn

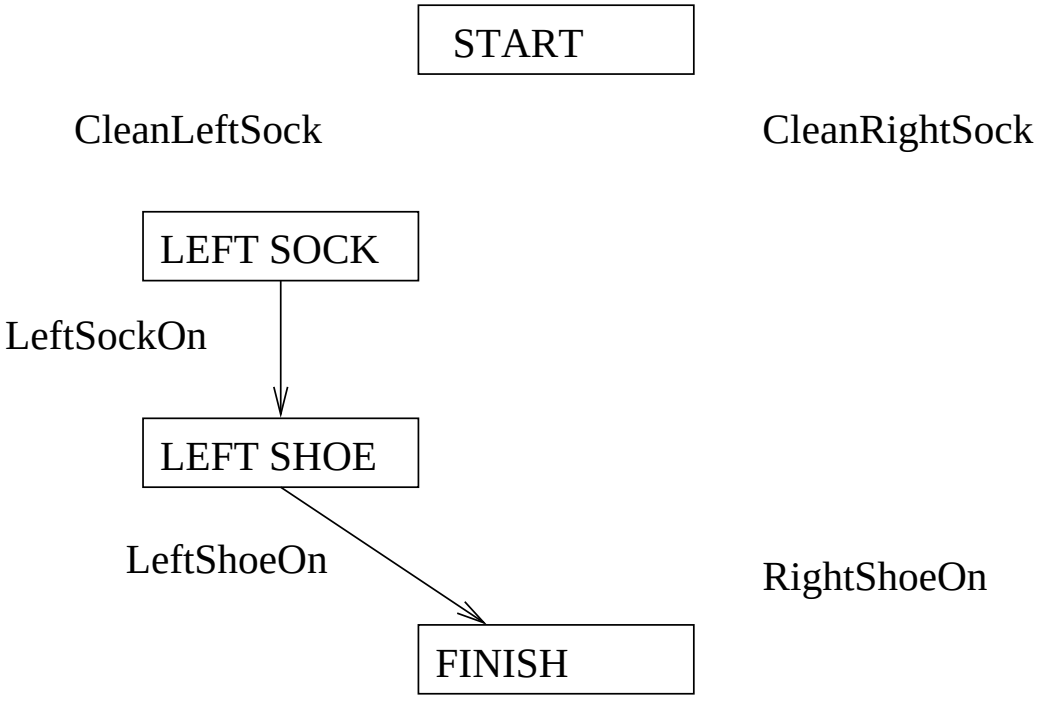
RightShoeOn

FINISH

# Example (cont'd)

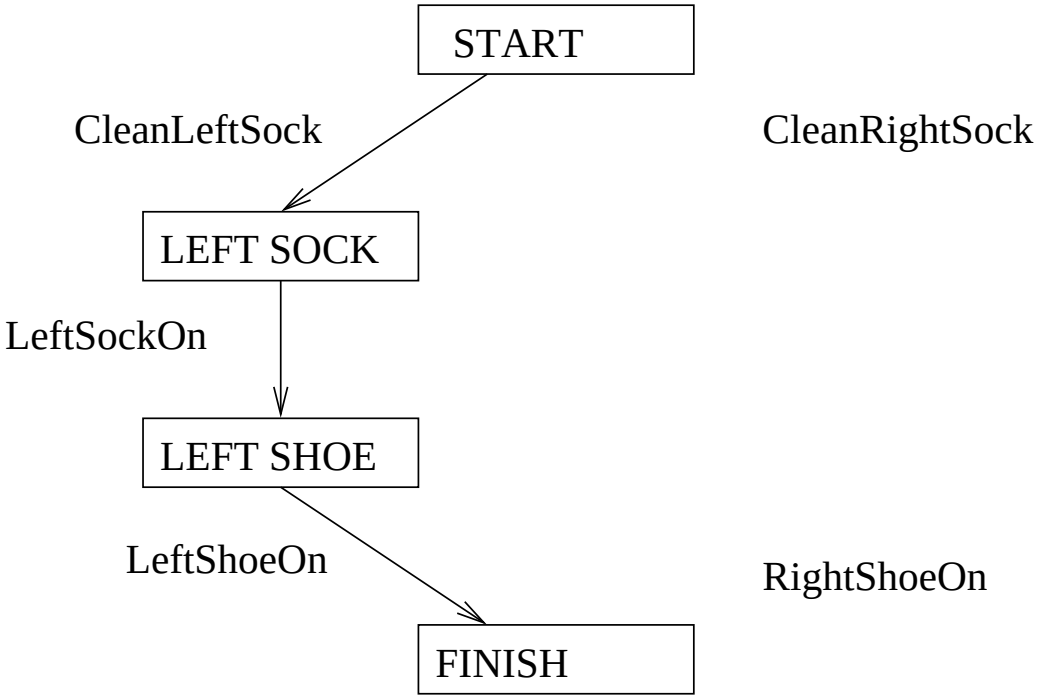


# Example (cont'd)




OC =
CleanLeftSock for LEFTSOCK
RightShoeOn for FINISH

# Example (cont'd)

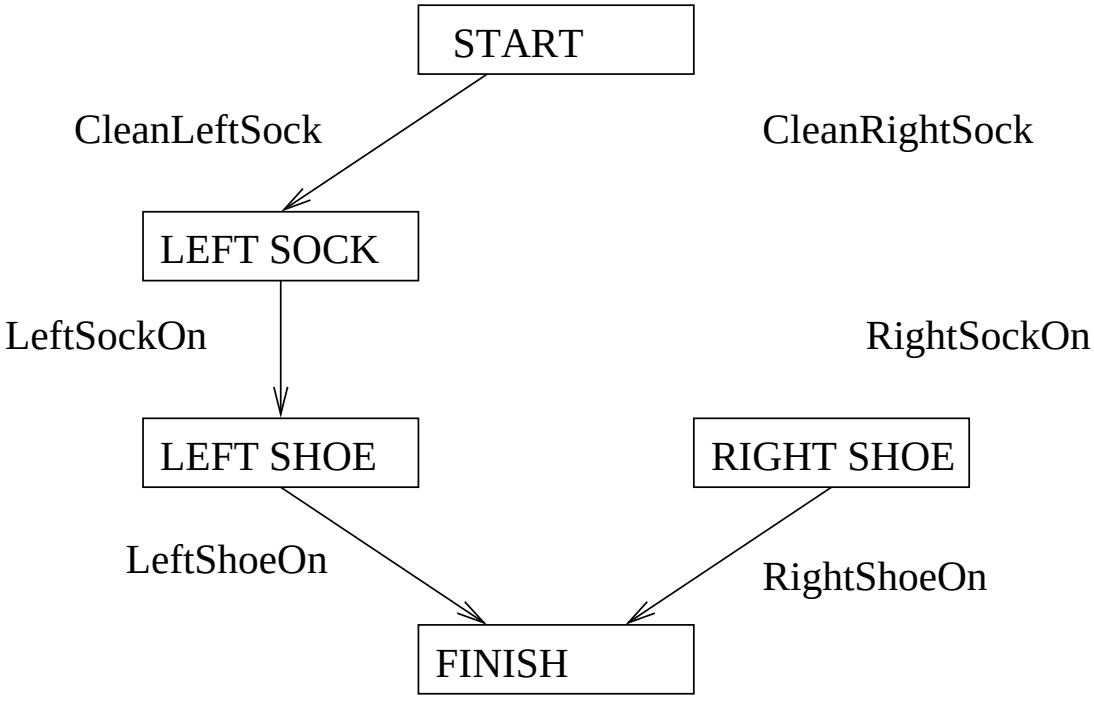



CleanRightSock

OC =
RightShoeOn for FINISH

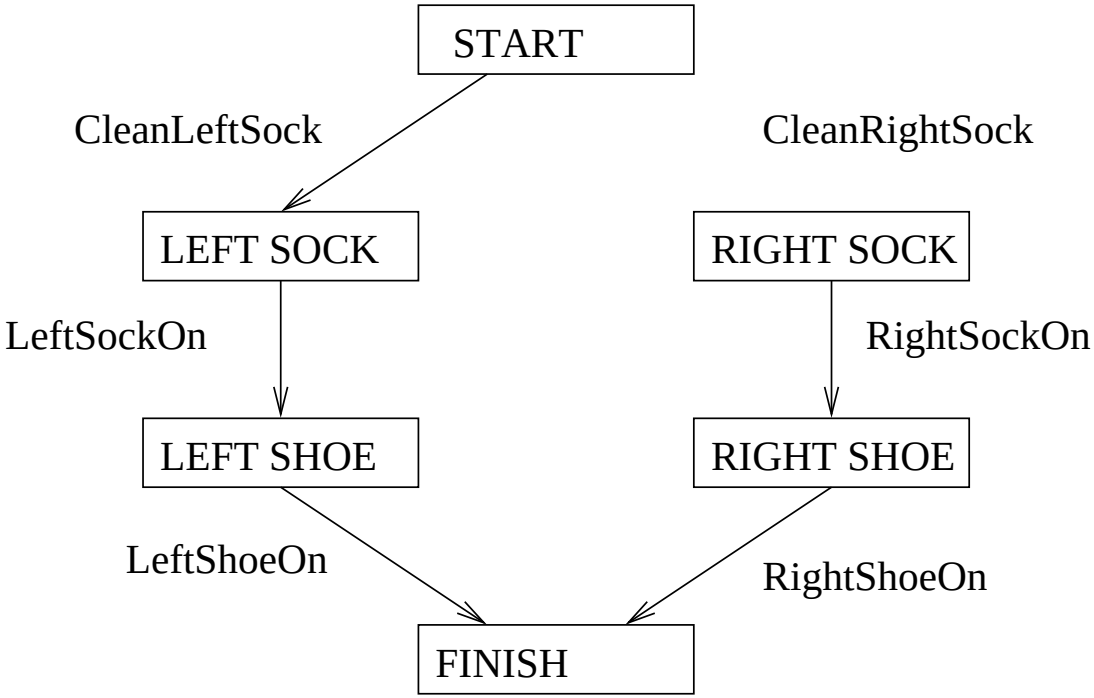
RightShoeOn

# Example (cont'd)



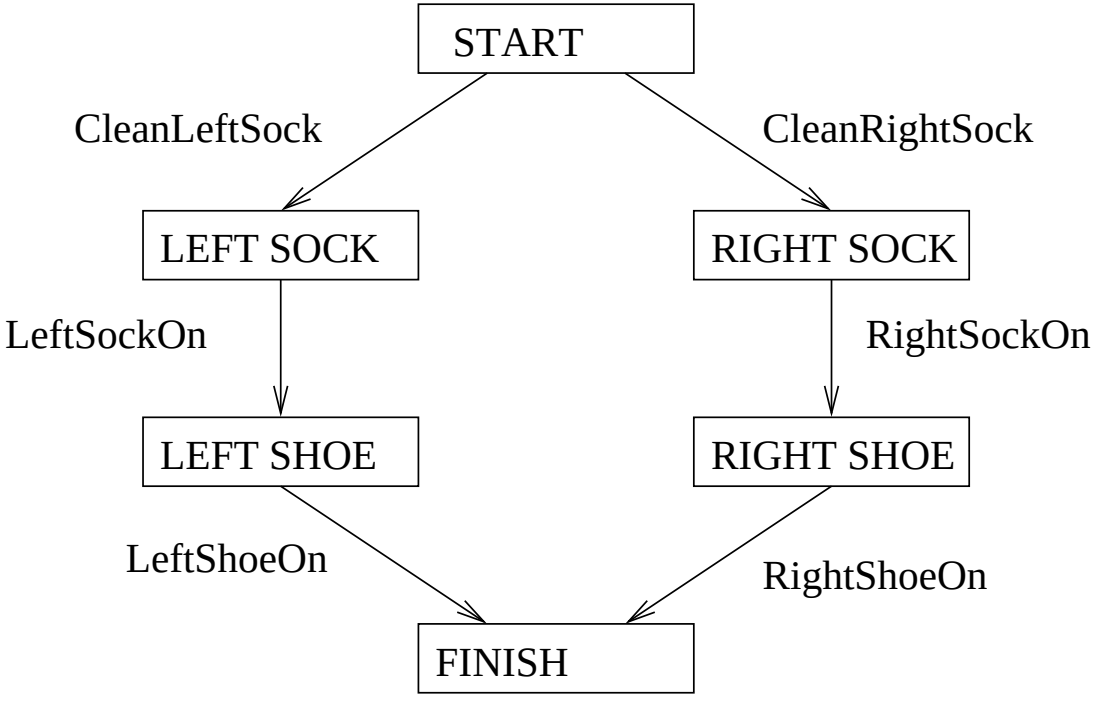

OC =
RightSockOn for RIGHTSHOE

# Example (cont'd)




OC =
CleanRightSock for RIGHTSOCK

# Example (cont'd)




OC=
{ }

# Planning process

Operators on partial plans:

- close open conditions:
  - add a link from an existing action to an open condition
  - add a step to fulfill an open condition
- resolve threats:
  - order one step wrt another to remove possible conflicts

Gradually move from incomplete/vague plans to complete, correct plans

Backtrack if an open condition is unachievable or if a conflict is unresolvable

# POP is a search in the plan space

```
function TREE-SEARCH (problem)
returns a solution, or failure

  initialize the frontier using the initial state of problem
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state
      then return the corresponding solution
    else expand the chosen node and add the resulting
      nodes to the frontier
  end
```

# POP algorithm specifics

The initial state, goal state and the operators are given. The planner converts them to required structures.

Initial state:
MAKE-MINIMAL-PLAN (*initial,goal*)

Goal-Test:
SOLUTION?(*plan*)

SOLUTION? returns true iff OC and UL are both empty.

Successor function:
The successors function could either close an open condition or resolve a threat.

# POP algorithm specifics (cont'd)

**function** SUCCESSORS (*plan*)
**returns** *a set of partially ordered plans*

*flaw-type* $\leftarrow$ SELECT-FLAW-TYPE (*plan*)
**if** *flaw-type* is an open condition **then**
$S_{need}, c \leftarrow$ SELECT-SUBGOAL (*plan*)
**return** CLOSE-CONDITION (*plan, operators,* $S_{need}$,*c*)
**if** *flaw-type* is a threat **then**
$S_{threat}, S_i, c, S_j \leftarrow$ SELECT-THREAT(*plan*)
**return** RESOLVE-THREAT (plan, $S_{threat}, S_i, c, S_j$)

# POP algorithm specifics (cont'd)

**function** CLOSE-CONDITION (*plan, operators,* $S_{need}$,*c*)
**returns** *a set of partially ordered plans*

*plans* $\leftarrow \emptyset$
**for each** $S_{add}$ from *operators* or STEPS(*plan*)
that has $c$ has an effect **do**
*new-plan* $\leftarrow$ plan
**if** $S_{add}$ is a newly added step from *operators* **then**
add $S_{add}$ to STEPS (*new-plan*)
add START $\prec S_{add} \prec$ FINISH to ORDERINGS (*new-plan*)
add the causal link $(S_{add}, c, S_{need})$ to LINKS (*new-plan*)
add the ordering constraint $(S_{add} \prec S_{need})$ to
ORDERINGS (*new-plan*)
add *new-plan* to *plans*
**end**

**return** *new-plans*

# POP algorithm specifics (cont'd)

**function** RESOLVE-THREAT (*plan*, $S_{threat}, S_i, c, S_j$)
**returns** *a set of partially ordered plans*

*plans* $\leftarrow \emptyset$
*//Demotion:*
*new-plan* $\leftarrow$ plan
add the ordering constraint $(S_{threat} \prec S_i)$ to ORDERINGS (*new-plan*)
**if** *new-plan* is consistent **then**
  add *new-plan* to *plans*
*//Promotion:*
*new-plan* $\leftarrow$ plan
add the ordering constraint $(S_j \prec S_{threat})$ to ORDERINGS (*new-plan*)
**if** *new-plan* is consistent **then**
  add *new-plan* to *plans*

**return** *new-plans*

# Shopping example

The operators are:
GO (?x, ?y)
  preconditions: at(?x)
  effects: ~at(?x), at(?y)

BUY (?s, ?i)
  preconditions: at(?s),
    ~bought(~i)
  effects: bought(?i)

START 0

at(H)
~bought(A)
~bought(B)

***bought(A)***
***bought(B)***
***at(H)***

FINISH f

Agenda:
  open subgoals:
    bought(A) for f
    bought(B) for f
    at(H) for f

The subgoals that are currently open are italicized.

add a causal link from START for at(H) for f

add a go(J,H) action for at(H) for f

INIT 0

at(H)

~bought(A)
~bought(B)

***bought(A)***
***bought(B)***

FINISH f

INIT 0

at(H)
~bought(A)
~bought(B)

***at(J)***

GO(J,H) 1

***bought(A)***
***bought(B)***

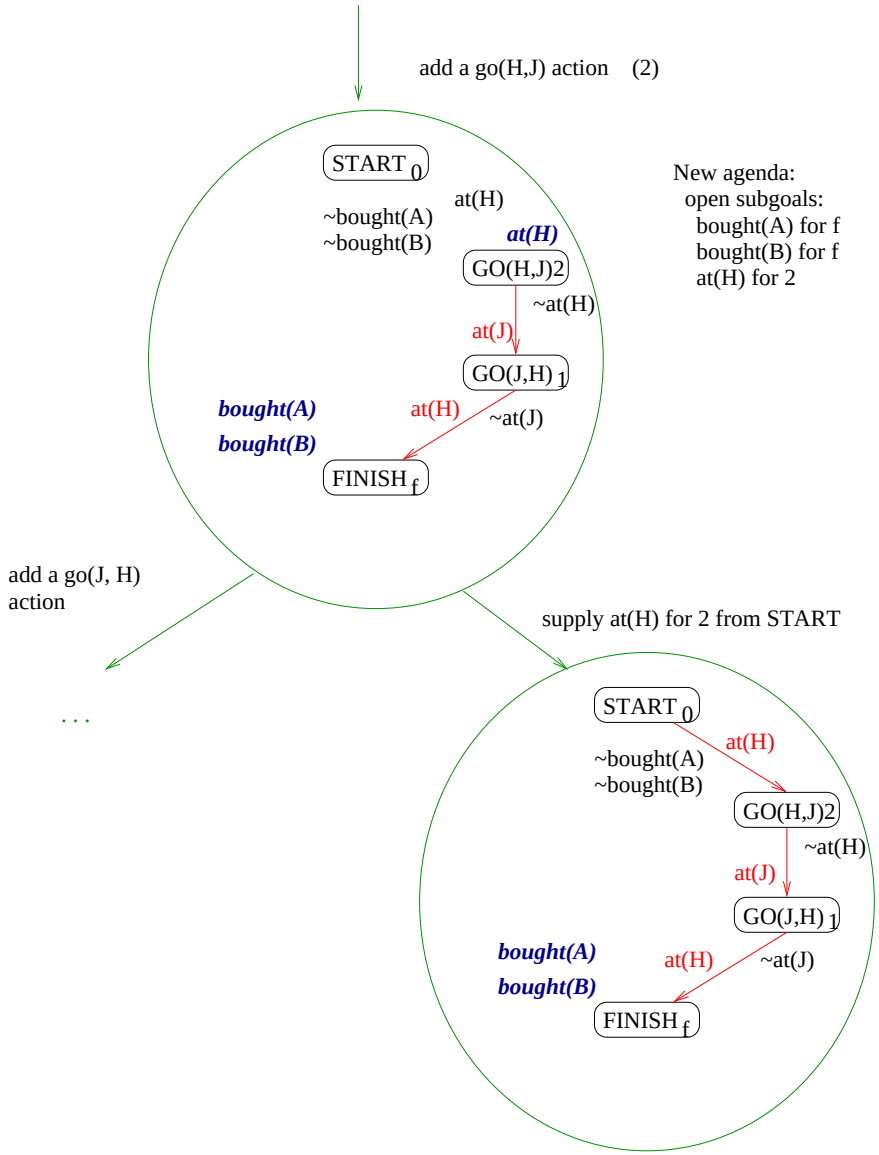
at(H)

~at(J)

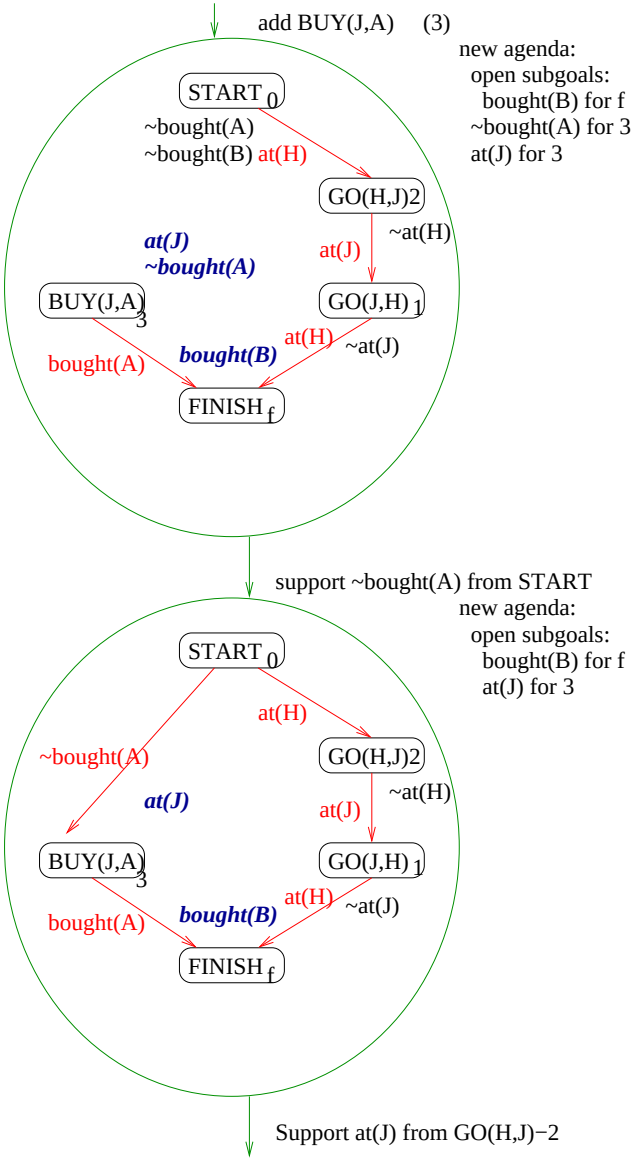
FINISH f

new Agenda:
  open subgoals:
    bought(A) for f
    bought(B) for f
    at(J) for 1

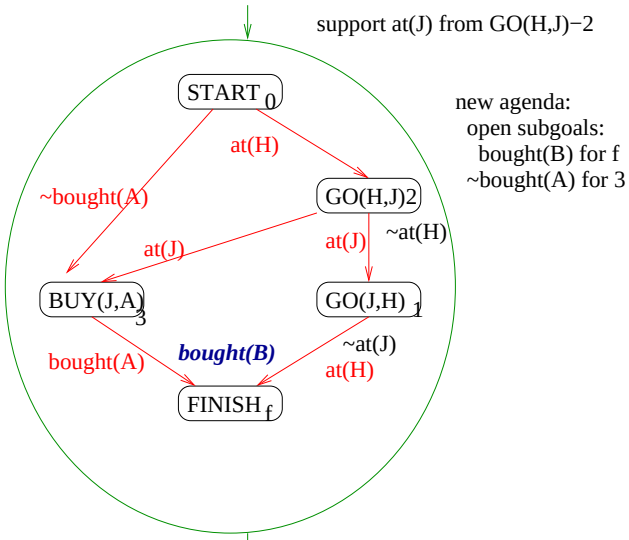
# Shopping example (cont'd)



# Shopping example (cont'd)

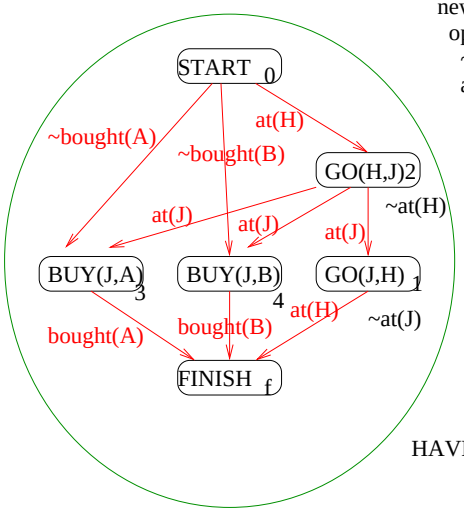


# Shopping example (cont'd)




new agenda:
open subgoals:
bought(B) for f
~bought(A) for 3

add BUY(J,B) (4)




new agenda:
open subgoals:
~bought(B) for 4
at(J) for 4

support ~bought(B) for 4 from START

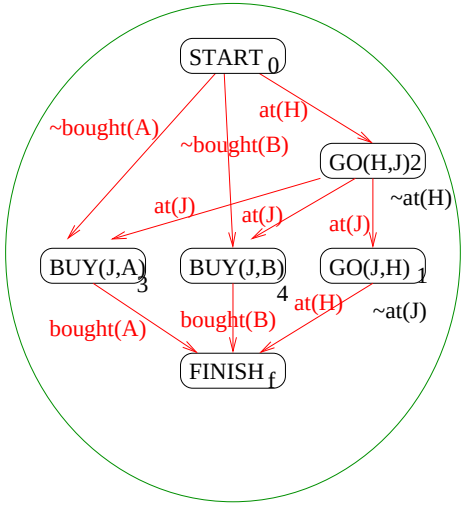
new agenda:
open subgoals:
at(J) for 4

support ~at(J) for 4
from GO(H,J)−2

new agenda:
open subgoals:
none

HAVEN'T CONSIDERED THE THREATS YET

# Shopping example (cont'd)

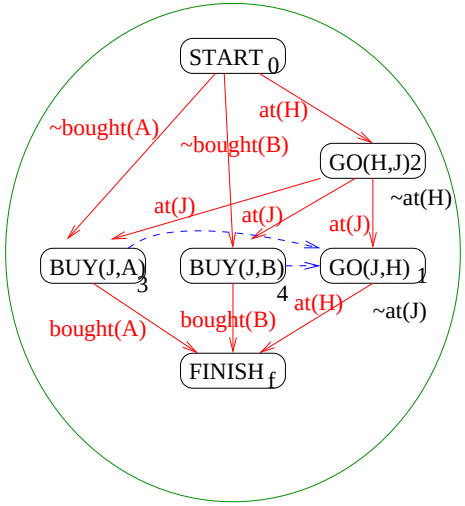



Now, the solution is a possible ordering of this plan. Those are:

2 3 4 1
2 3 1 4
2 4 3 1
2 4 1 3
2 1 3 4
2 1 4 3

It should not be possible to order GO(J,H) before any of the BUY actions.

# Shopping example (cont'd)

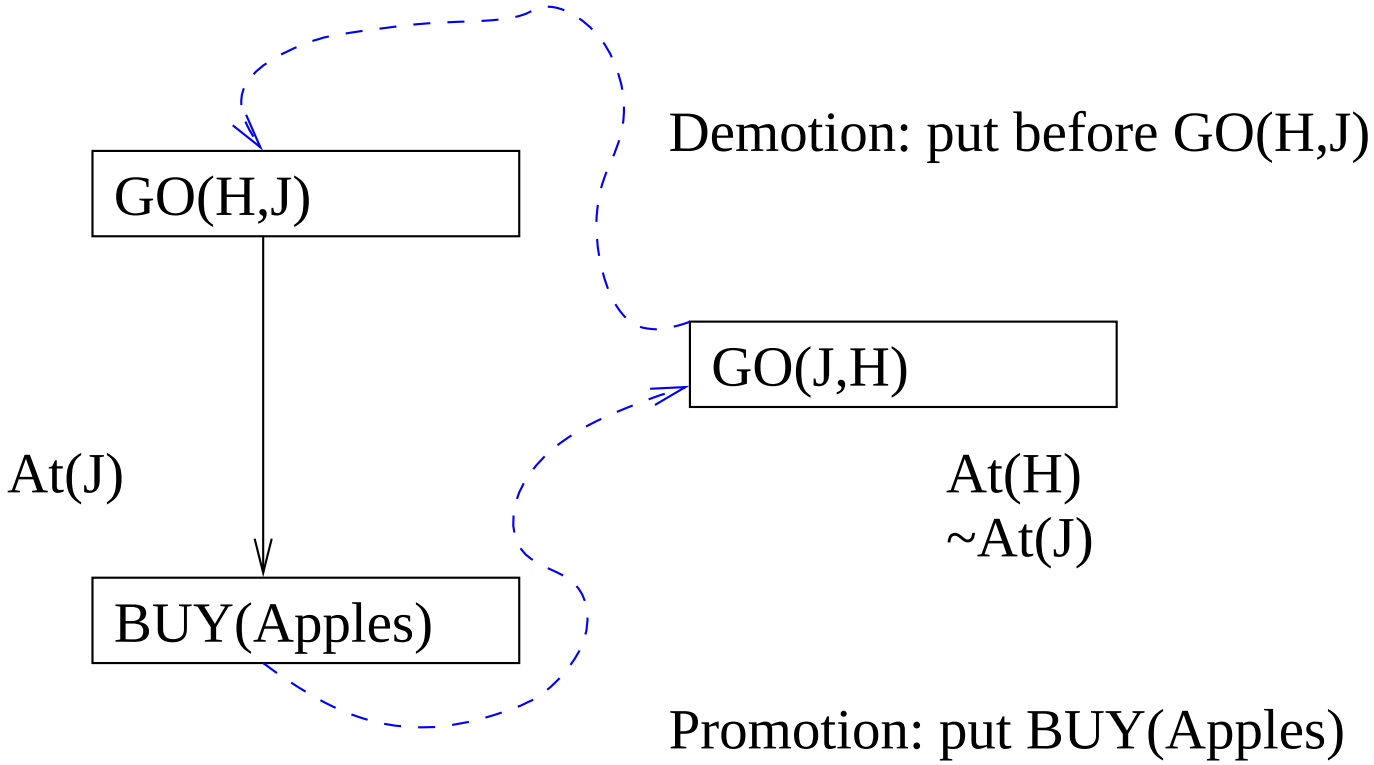



This is a correct partially ordered plan.
It is complete.
The possible total orders are:
2 3 4 1
2 4 3 1

The agent has to go to Jim's first.
It order of getting the items does not matter.
Then it has to go back home.

# Threats and promotion/demotion

A threatening step is a potentially intervening step that destroys the condition achieved by a causal link. E.g., GO(J,H) threatens At(J)



# Properties of POP

- Nondeterministic algorithm: backtracks at choice points on failure:
  - choice of $S_{add}$ to achieve $S_{need}$
  - choice of demotion or promotion for threat resolution
  - selection of $S_{need}$ is irrevocable
- POP is sound, complete, and systematic (no repetition)
- Extensions for disjunction, universals, negation, conditionals
- Particularly good for problems with many loosely related subgoals

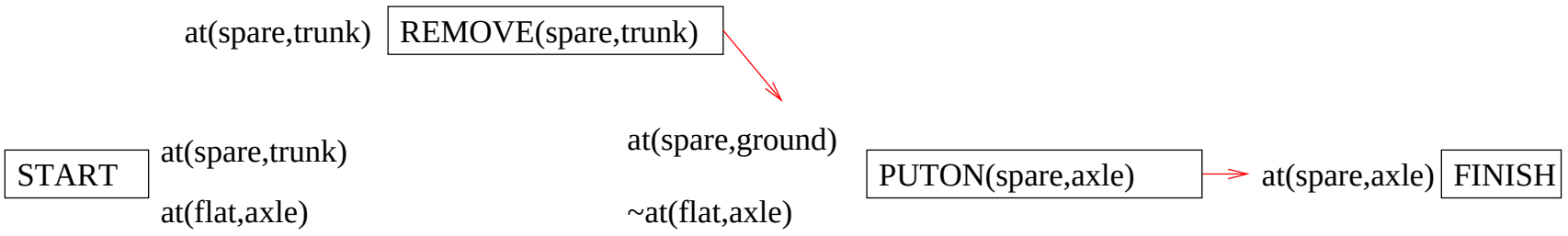
# Additional POP examples

- The flat tire example shows the effect of inserting an “impossible” action.
- The Sussman anomaly shows that “divide-and-conquer” is not always optimal. POP can find the optimal plan.

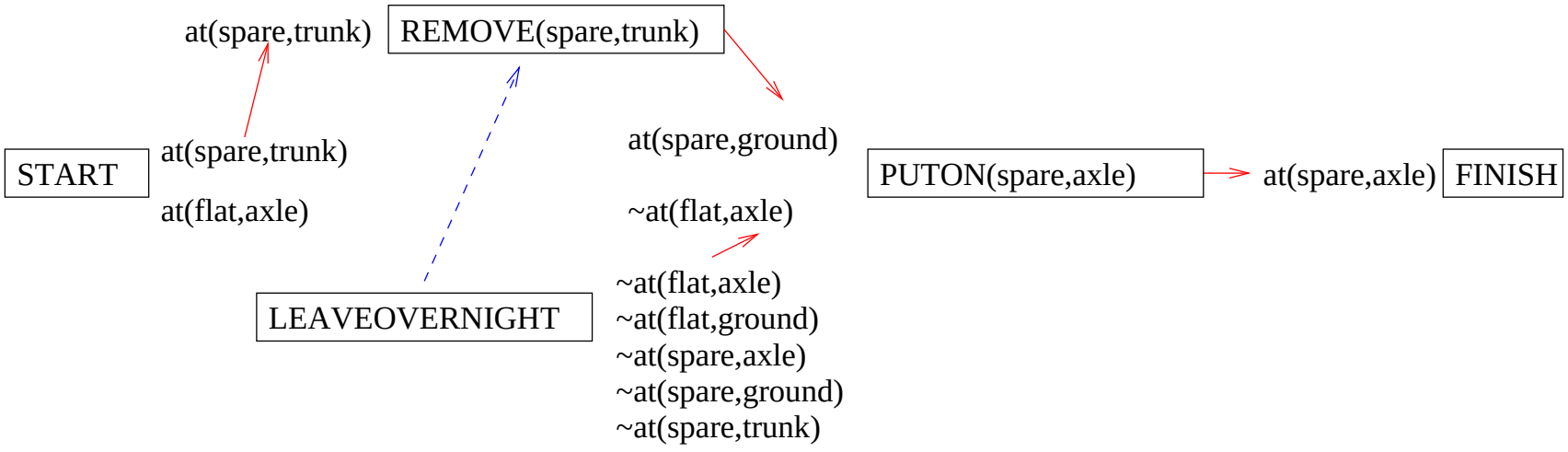
# The flat tire domain

Init(At(Flat,Axle) $\wedge$ At(Spare,Trunk))
Goal(At(Spare,Axle))
Action(REMOVE(spare,trunk),
    Precond: At(spare,trunk)
    Effect: $\neg$At(spare,trunk) $\wedge$ At(spare,ground)
Action(REMOVE(flat,axle),
    Precond: At(flat,axle)
    Effect: $\neg$At(flat,axle) $\wedge$ At(flat,ground)
Action(PUTON(spare,axle),
    Precond: At(spare,ground) $\wedge$ $\neg$ at(flat,axle)
    Effect: $\neg$At(spare,ground) $\wedge$ At(spare,axle)
Action(LEAVEOVERNIGHT
    Precond:
    Effect: $\neg$At(spare,ground) $\wedge$ $\neg$ At(spare,axle)
            $\neg$At(spare,trunk) $\wedge$ $\neg$ At(flat,ground)
            $\neg$At(flat,axle)

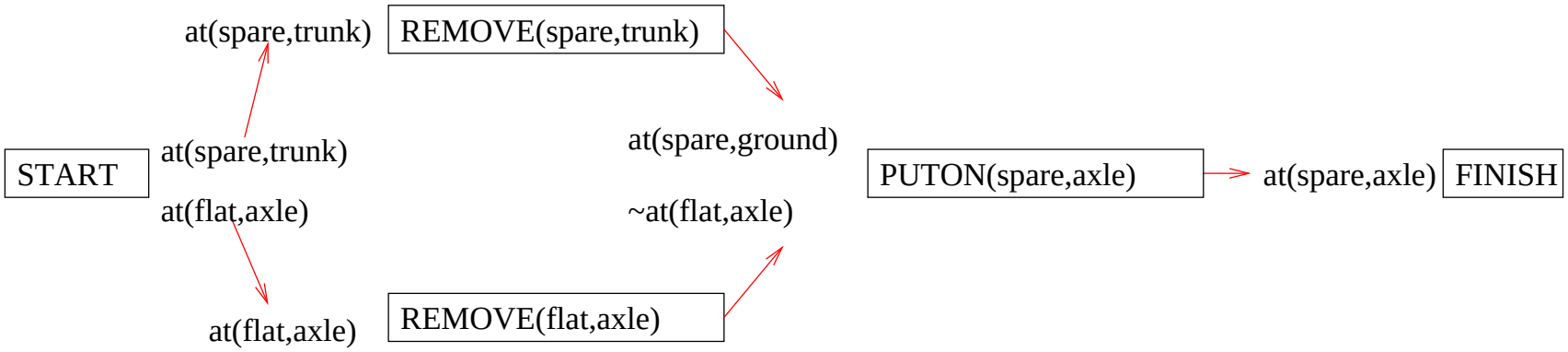
# The flat tire plan



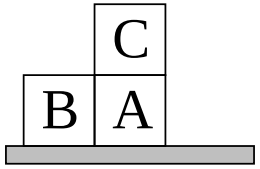
# The flat tire plan (cont'd)



# The flat tire plan (cont'd)



# Sussman anomaly




Clear(x)  On(x,z)  Clear(y)

PUTON(x,y)

~On(x,z)  ~Clear(y)

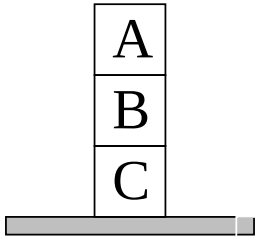
Clear(z)  On(x,y)

Clear(x)  On(x,z)

PUTONTABLE(x)

~On(x,z)

Clear(z)  On(x,Table)

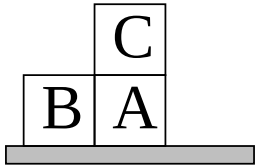



+ several inequality constraints

# Sussman anomaly (cont'd)

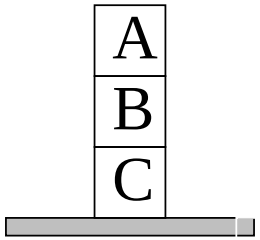
START

On(C,A) On(A,Table) Clear(B) On(B,Table) Clear(C)




On(A,B) On(B,C)

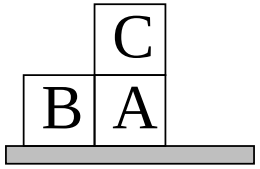
FINISH

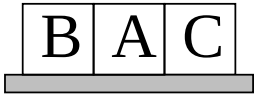


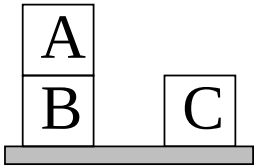
# Sussman anomaly (cont'd)

START

On(C,A) On(A,Table) Clear(B) On(B,Table) Clear(C)



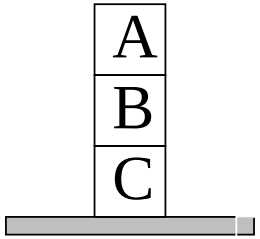




If we try the first goal ( on(A,B) ) first,
we can't proceed without undoing work

On(A,B) On(B,C)

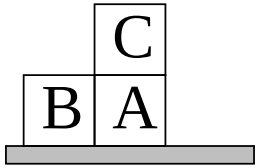
FINISH

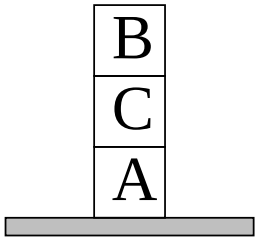


# Sussman anomaly (cont'd)

START

On(C,A) On(A,Table) Clear(B) On(B,Table) Clear(C)

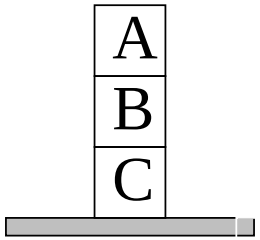



If we try the second goal ( on (B,C) ) first,
we can't proceed without undoing work.

On(A,B) On(B,C)

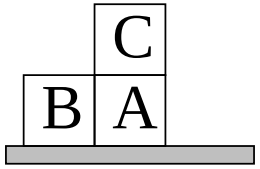
FINISH



# Sussman anomaly (cont'd)

START

On(C,A) On(A,Table) Clear(B) On(B,Table) Clear(C)

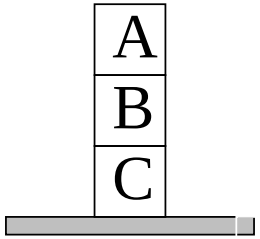



Clear(B) On(B,z) Clear(C)

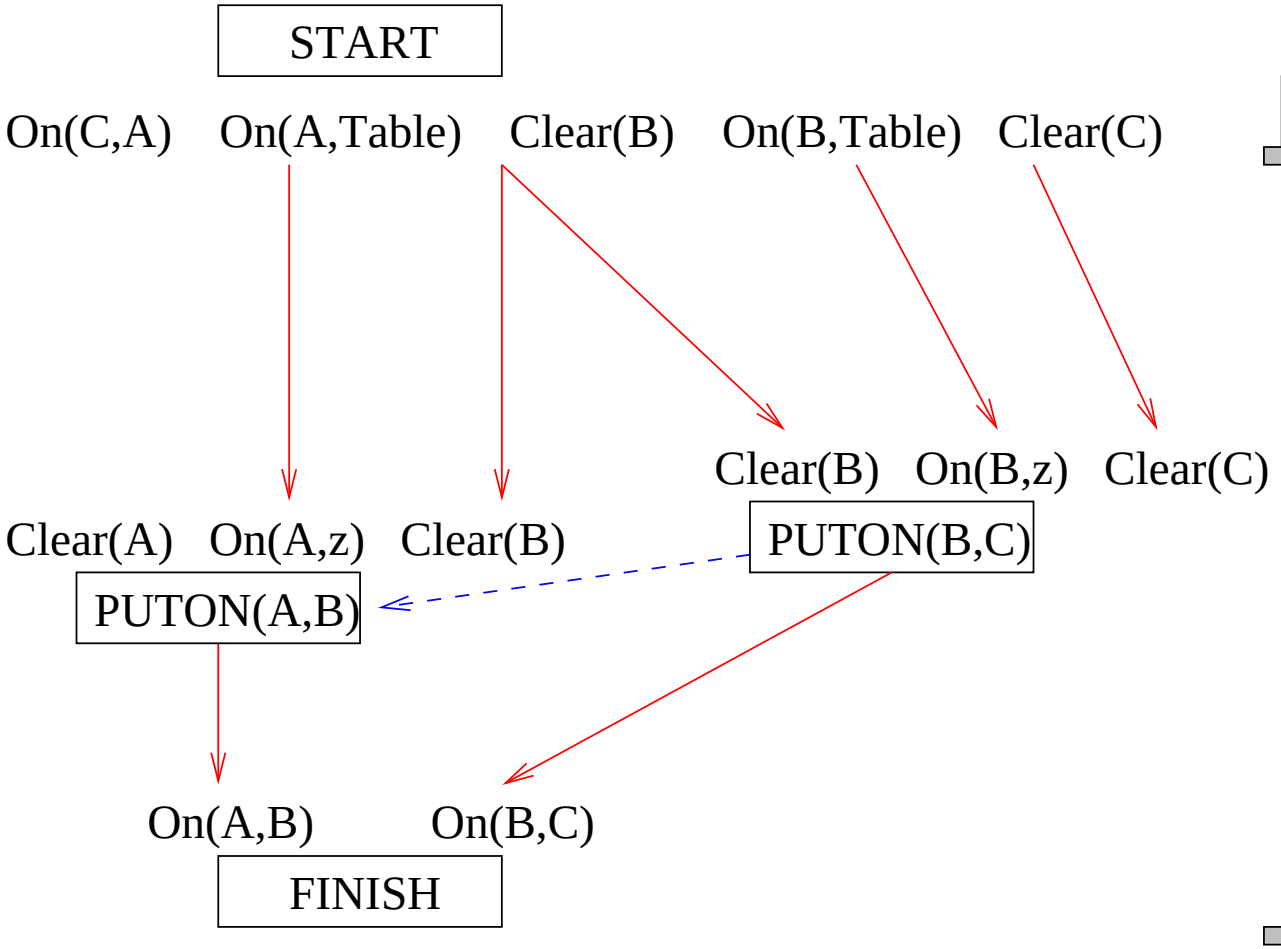
PUTON(B,C)

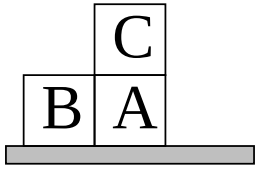
On(A,B) On(B,C)

FINISH



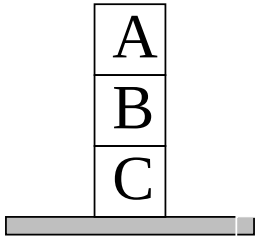
# Sussman anomaly (cont'd)



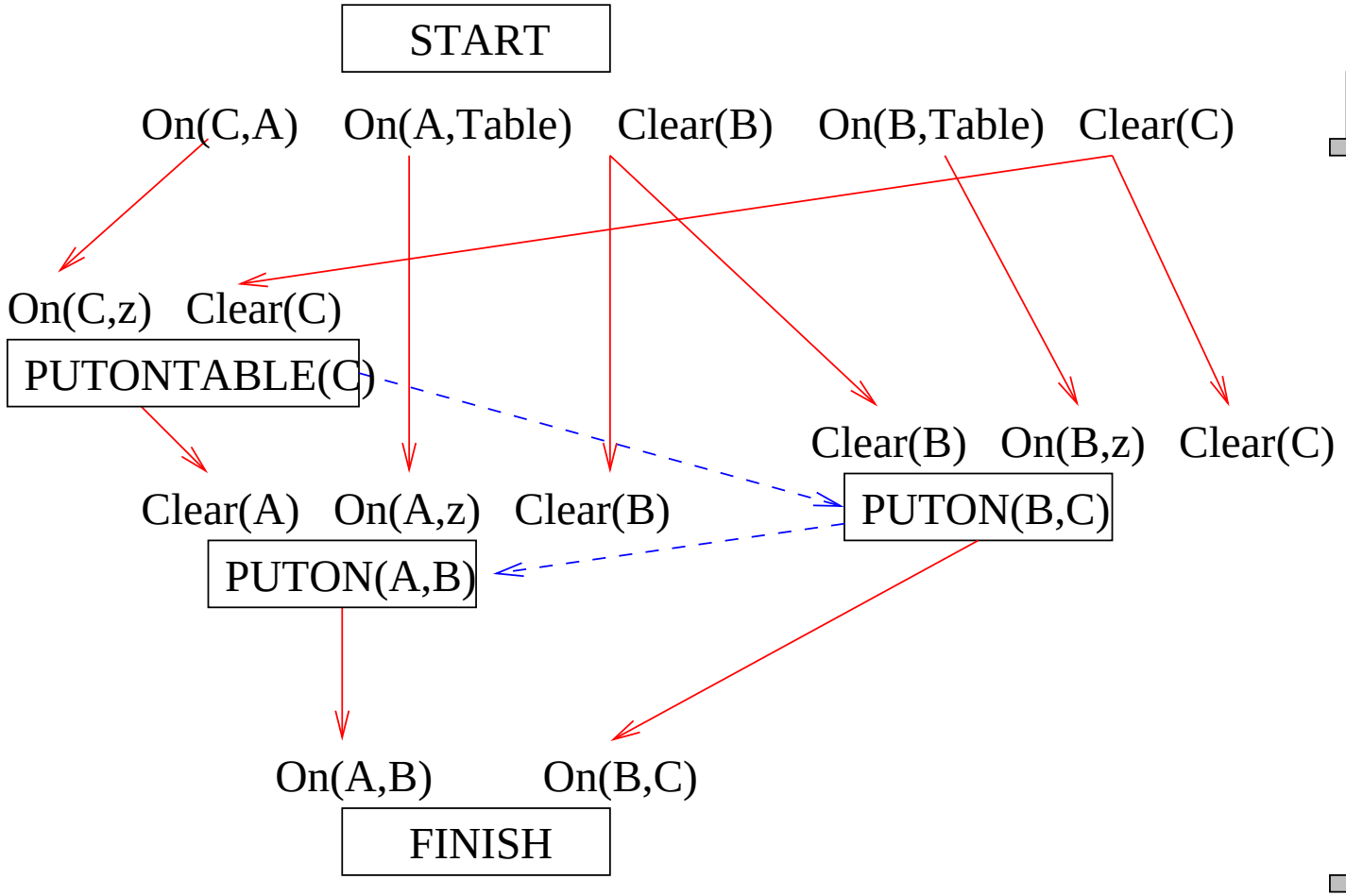



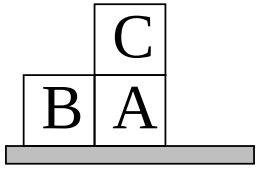

PUTON(A,B) threatens Clear(B)

Order after PUTON(B,C)



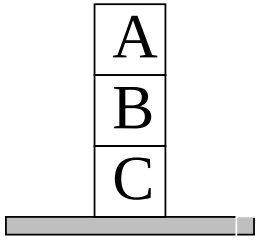
# Sussman anomaly (cont'd)




PUTON(B,C) threatens Clear(C)

order after PUTON(A,B)



# Applications of planning

- **Robotics**
  - **Shakey, the robot at SRI was the initial motivator.**
  - **However, several other techniques are used for path-planning etc.**
  - **Most robotic systems are *reactive.***

- **Games**
  **The story is a plan and a different one can be constructed for each game.**

- **Web applications**
  **Formulating query plans, using web services.**

- **Crisis response**
  **Oil spill, forest fire, emergency evacuation.**

# **Applications of planning** (cont'd)

- **Space**
  **Autonomous spacecraft, self-healing systems.**

- **Device control**
  **Elevator control, control software for modular devices.**

- **Military planning.**

- **And many others.**

# Comments on planning

- It is a synthesis task.
- Classical planning is based on the assumptions of a deterministic and static environment.
- Theorem proving and situation calculus are not widely used nowadays for planning (see below).
- Algorithms to solve planning problems include:
  - forward chaining: heuristic search in state space
  - Graphplan: mutual exclusion reasoning using plan graphs
  - Partial order planning (POP): goal directed search in plan space
  - Satifiability based planning: convert problem into logic

# **Comments on planning** (cont'd)

- **Non-classical planners include:**
  - **probabilistic planners**
  - **contingency planners (a.k.a. conditional planners)**
  - **decision-theoretic planners**
  - **temporal planners**
  - **resource based planners**

# **Comments on planning** (cont'd)

- **In addition to plan generation algorithms we also need algorithms for**
  - **Carrying out the plan**
  - **Monitoring the execution**
    **(because the plan might not work as expected; or the world might change)**
    **(need to maintain the consistency between the world and the program's internal model of the world)**
  - **Recovering from plan failures**
  - **Acting on new opportunities that arise during execution**
  - **Learning from experience**
    **(save and generalize good plans)**

# Additional Reading

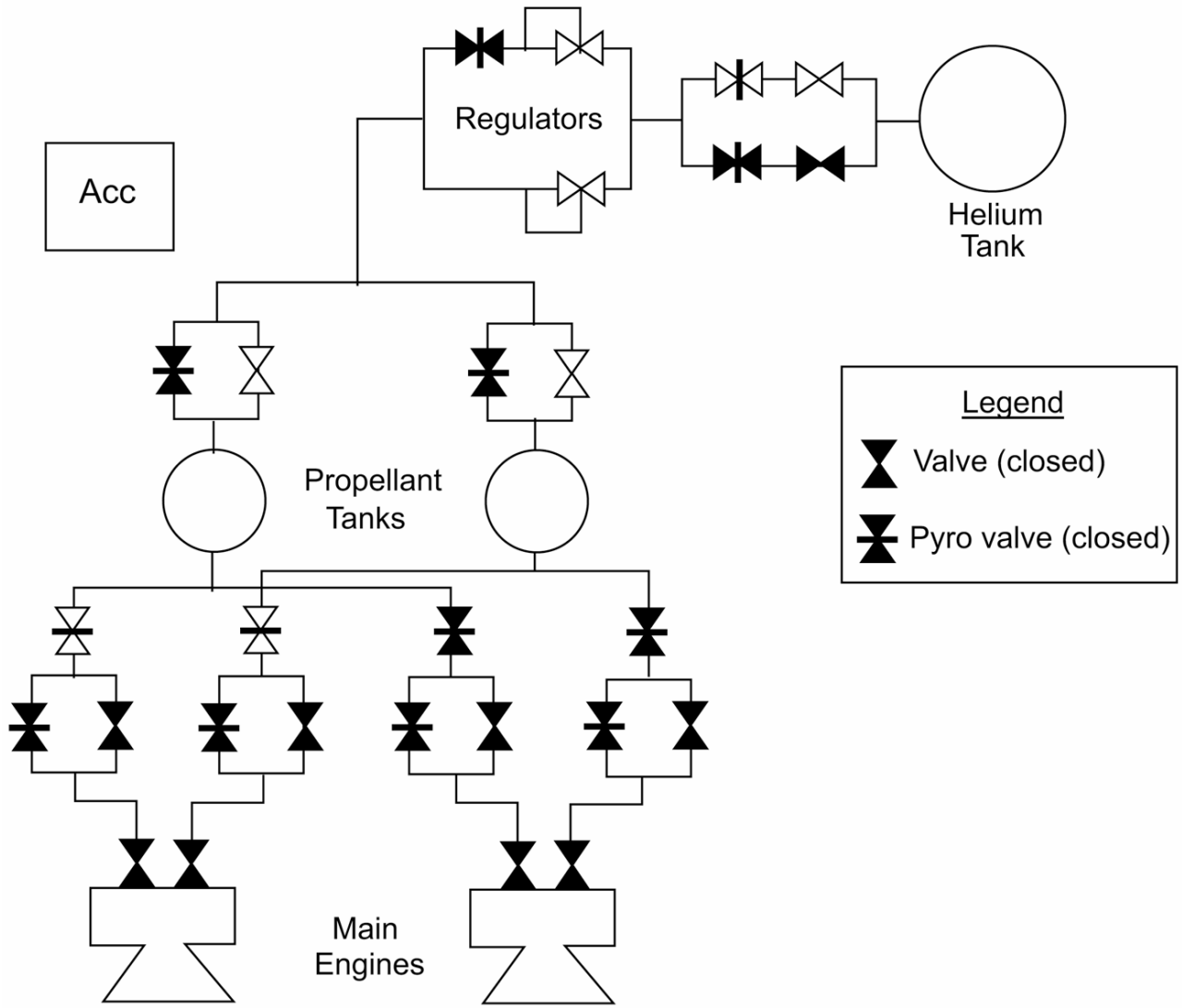
# **Model-based reactive configuration management** (Williams and Nayak, 1996a)

**Intelligent space probes that autonomously explore the solar system.**

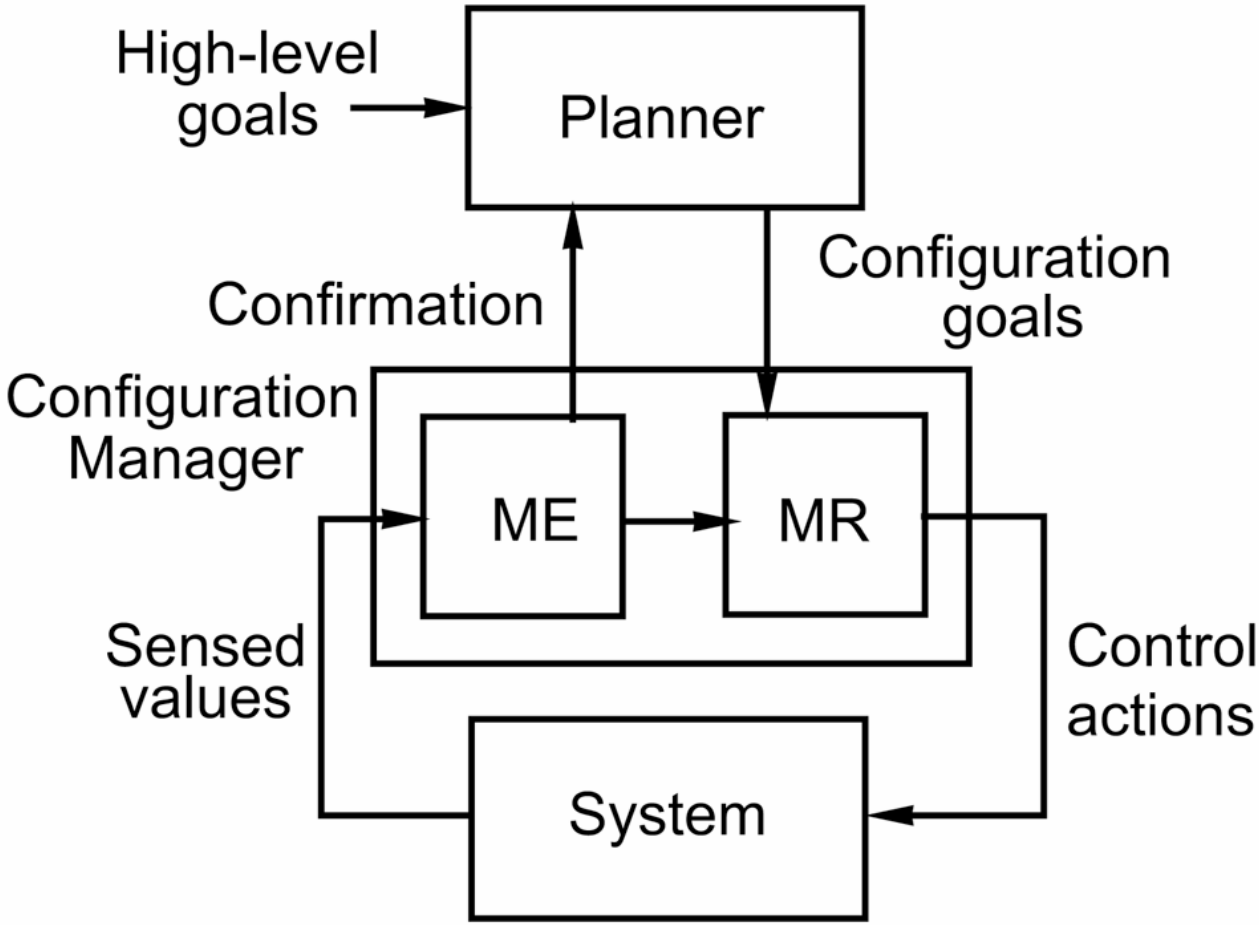
**The spacecraft needs to:**

- **radically reconfigure its control regime in response to failures,**
- **plan around these failures during its remaining flight.**

# A schematic of the simplified Livingstone propulsion system (Williams and Nayak ,1996)

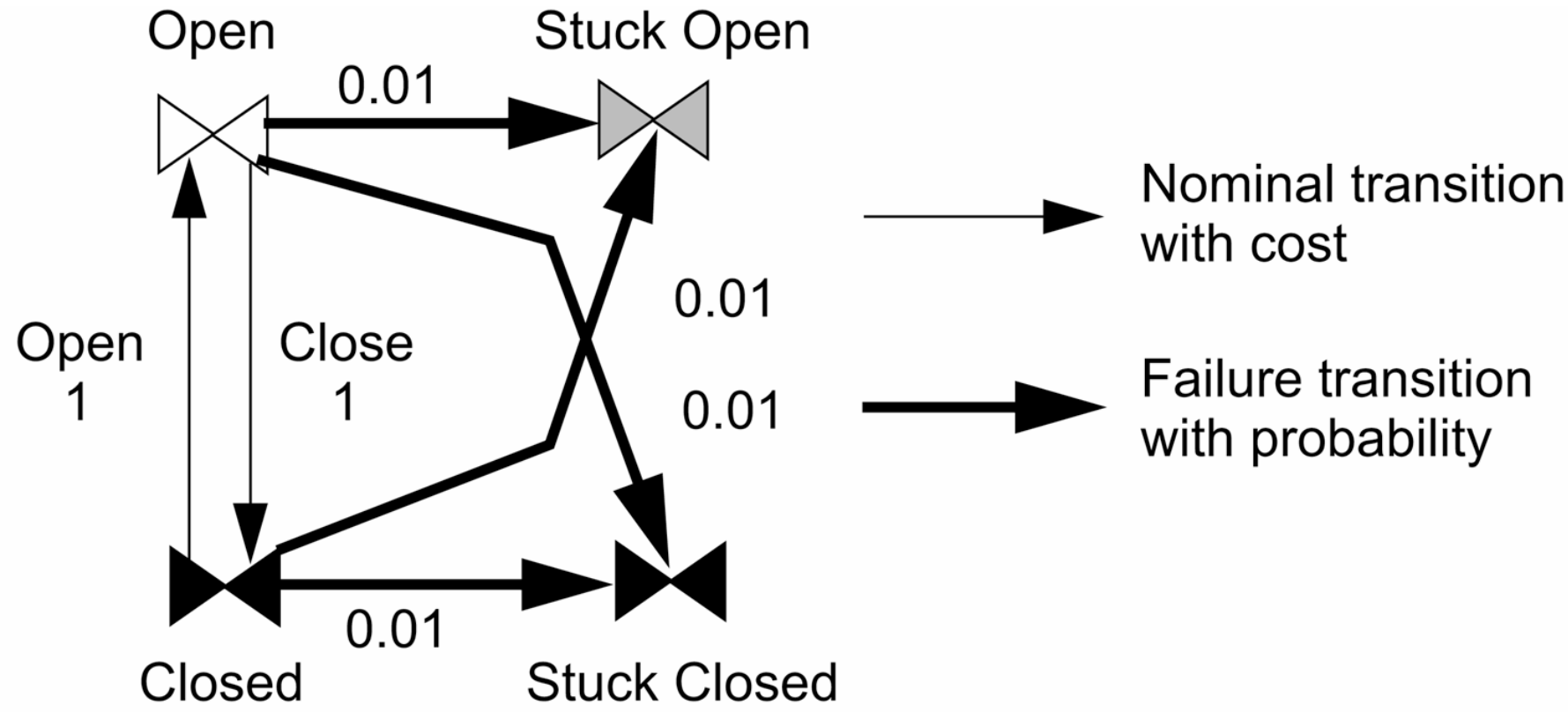


# **A model-based configuration management system** (Williams and Nayak, 1996)

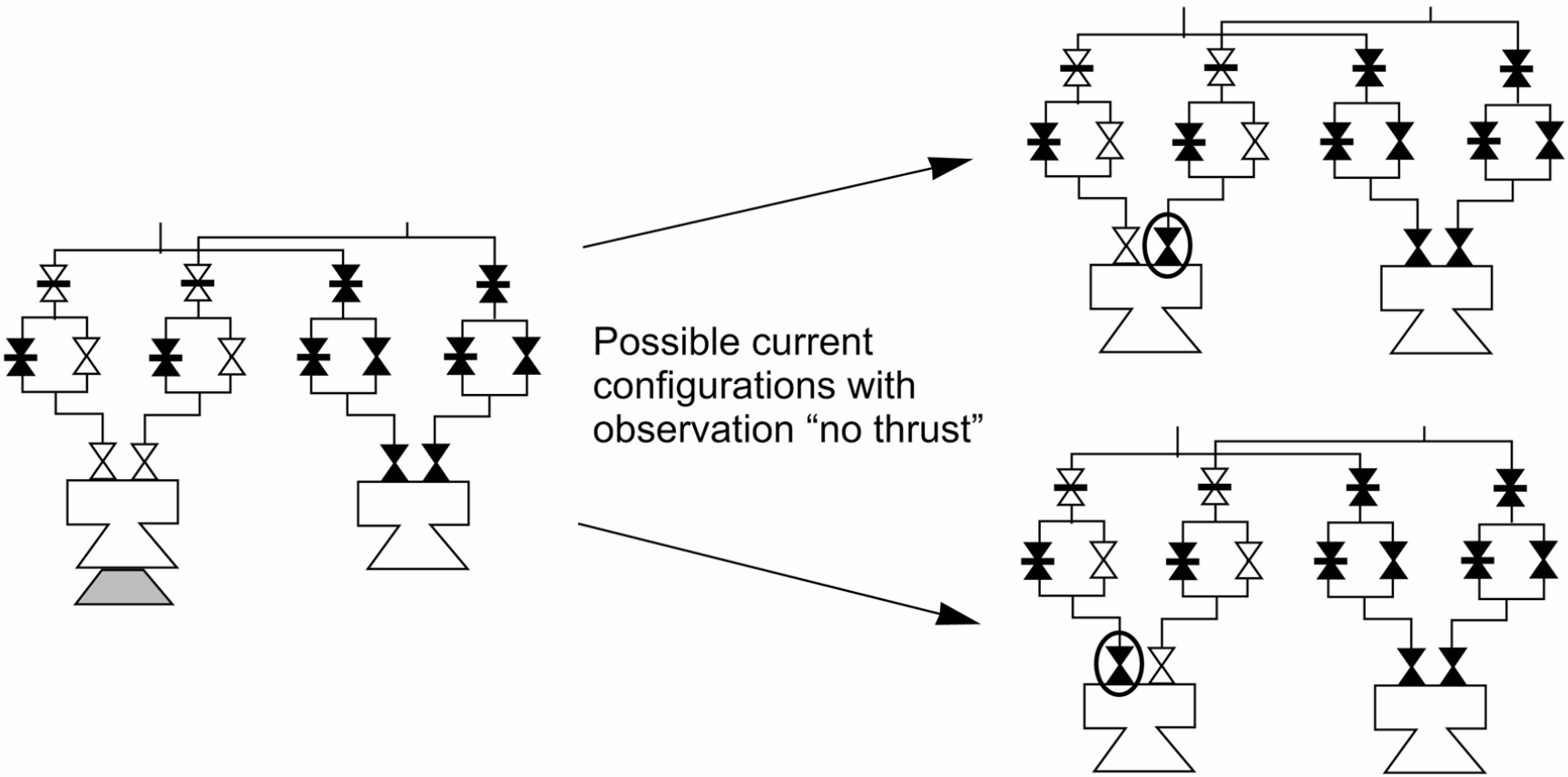



**ME: mode estimation** **MR: mode reconfiguration**

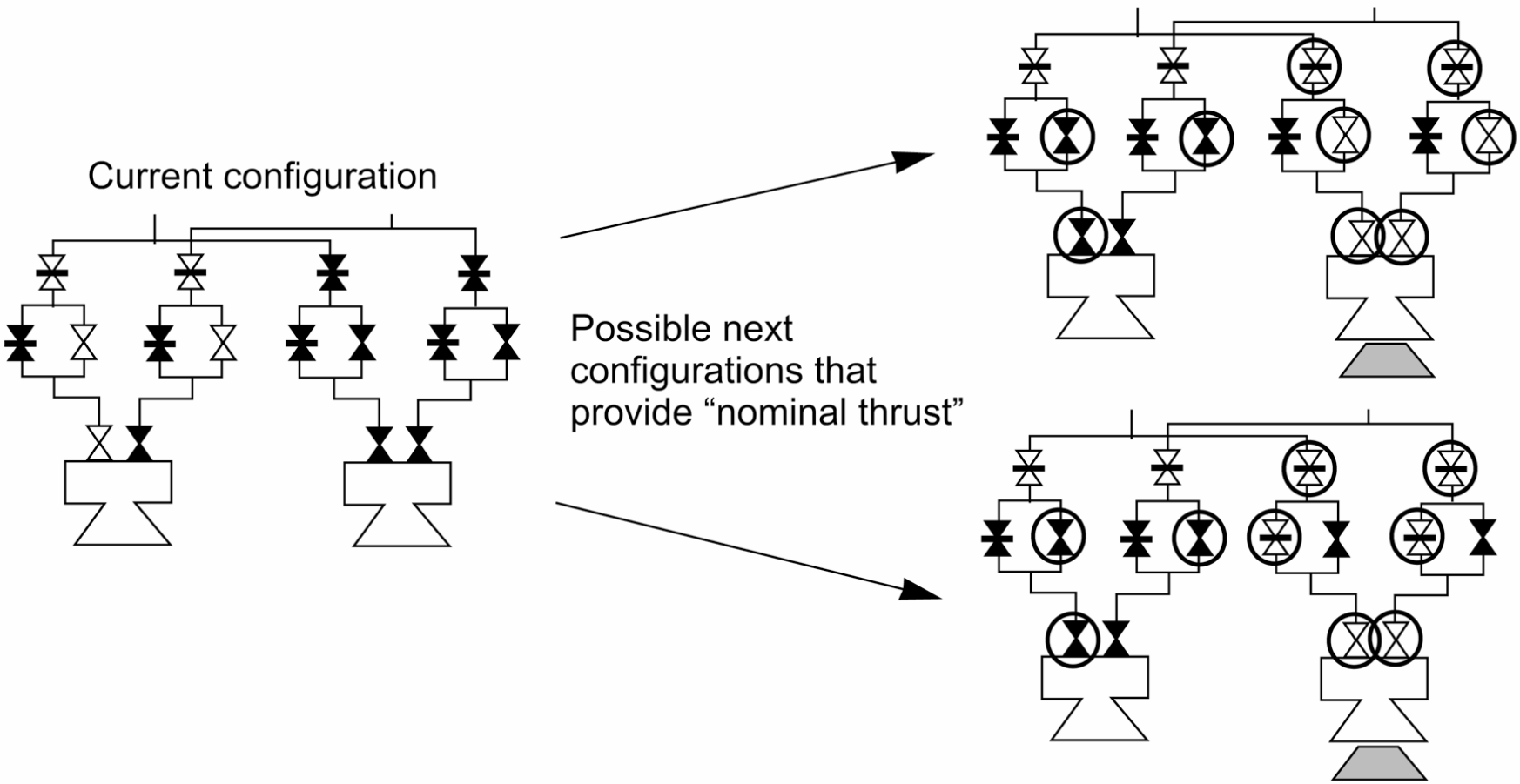
# The transition system model of a valve
(Williams and Nayak, 1996a)



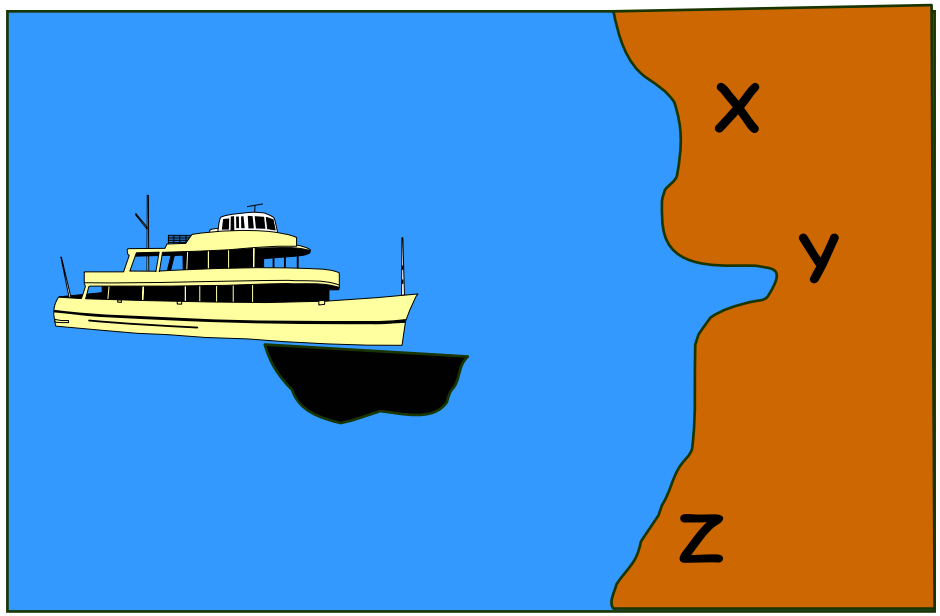
# **Mode estimation** (Williams and Nayak, 1996a)



# **Mode reconfiguration (MR)**
(Williams and Nayak, 1996a)



# Oil spill response planning

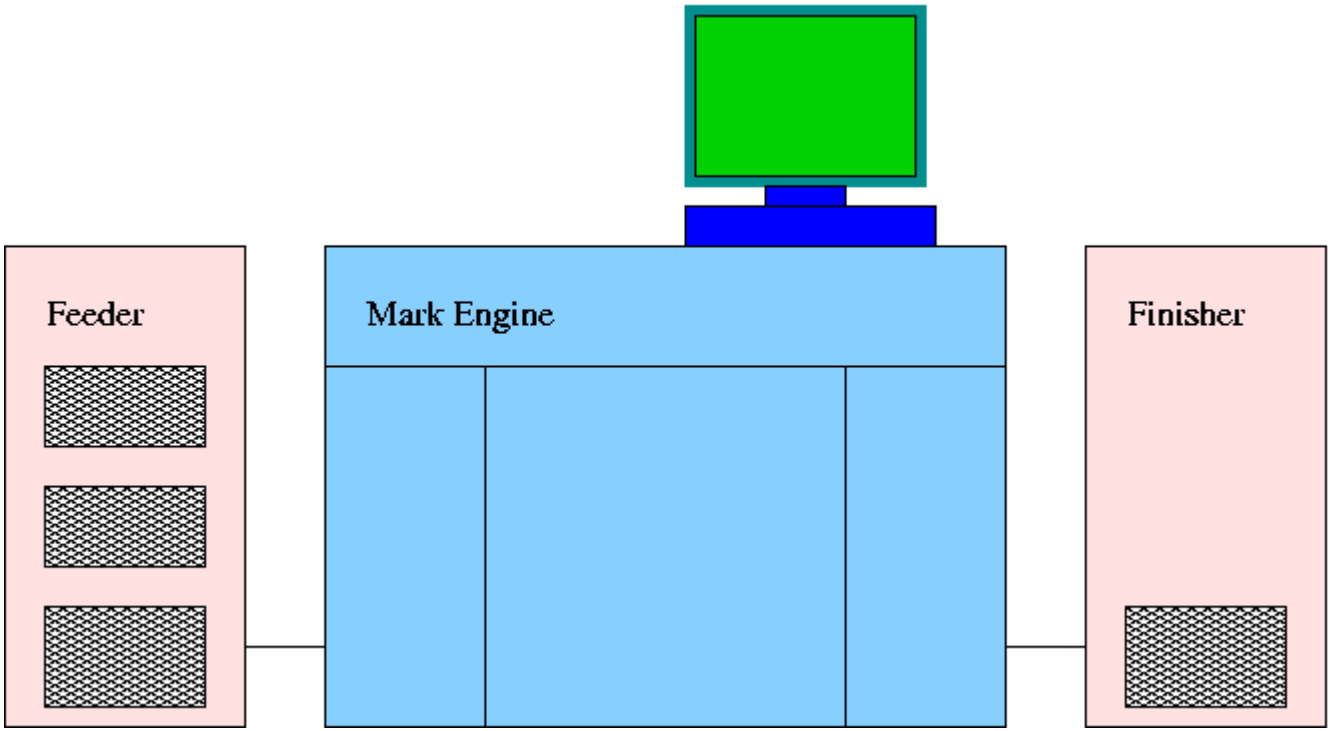



**(Desimone & Agosto 1994)**

**Main goals: stabilize discharge, clean water, protect sensitive shore areas**

**The objective was to estimate the equipment required rather than to execute the plan**

# A modern photocopier

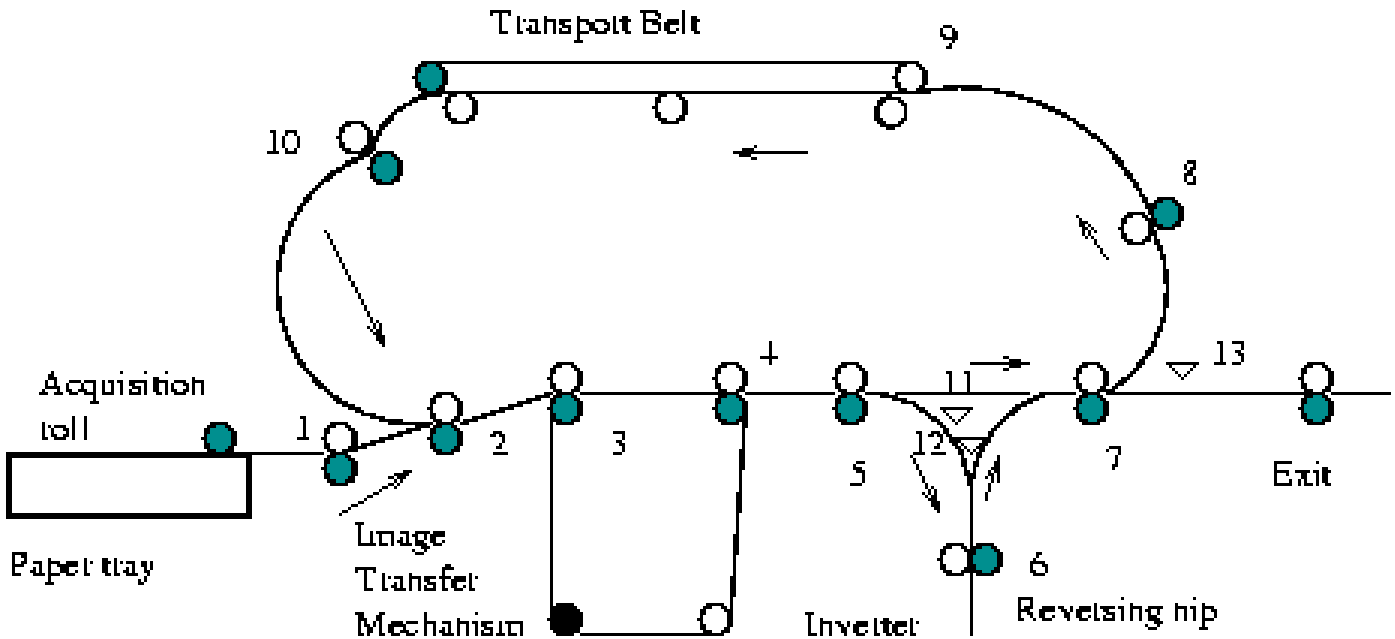



**(From a paper by Fromherz et al. 2003)**

**Main goal: produce the documents as requested by the user**

**Rather than writing the control software, write a controller that produces and executes plans**

# The paper path



# Monkeys can plan

**Why shouldn't computers?**

**The problem statement: A monkey is in a laboratory room containing a box, a knife and a bunch of bananas. The bananas are hanging from the ceiling out of the reach of the monkey. How can the monkey obtain the bananas?**

# VHPOP coding of the monkey and bananas problem

```
(define (domain monkey-domain)
  (:requirements :equality)
  (:constants monkey box knife glass water
    waterfountain)
  (:predicates (on-floor) (at ?x ?y) (onbox ?x)
    (hasknife) (hasbananas) (hasglass) (haswater)
    (location ?x)

  (:action go-to
    :parameters (?x ?y)
    :precondition (and (not = ?y ?x)) (on-floor)
      (at monkey ?y)
    :effect (and (at monkey ?x) (not (at monkey ?y))))
```

# **VHPOP coding** (cont'd)

```
(:action climb
  :parameters (?x)
  :precondition (and (at box ?x) (at monkey ?x))
  :effect (and (onbox ?x) (not (on-floor))))

(:action push-box
  :parameters (?x ?y)
  :precondition (and (not (= ?y ?x)) (at box ?y)
    (at monkey ?y) (on-floor))
  :effect (and (at monkey ?x) (not (at monkey ?y))
    (at box ?x) (not (at box ?y))))
```

# **VHPOP coding** (cont'd)

```
(:action get-knife
  :parameters (?y)
  :precondition (and (at knife ?y) (at monkey ?y))
  :effect (and (hasknife) (not (at knife ?y))))

(:action grab-bananas
  :parameters (?y)
  :precondition (and (hasknife) (at bananas ?y)
    (onbox ?y) )
  :effect (hasbananas))
```

# **VHPOP coding** (cont'd)

```
(:action pick-glass
  :parameters (?y)
  :precondition (and (at glass ?y) (at monkey ?y))
  :effect (and (hasglass) (not (at glass ?y))))

(:action get-water
  :parameters (?y)
  :precondition (and (hasglass) (at waterfountain ?y)
    (ay monkey ?y) (onbox ?y))
  :effect (haswater))
```

# Problem 1: monkey-test1.pddl

```
(define (problem monkey-test1)
  (:domain monkey-domain)
  (:objects p1 p2 p3 p4)
  (:init (location p1) (location p2)
         (location p3) (location p4)
         (at monkey p1) (on-floor)
         (at box p2) (at bananas p3)
         (at knife p4))
  (:goal (hasbananas)))
```

**go-to p4 p1**
**get-knife p4**
**go-to p2 p4**
**push-box p3 p2**
**climb p3**
**grab-bananas p3** **time = 30 msec.**

# Problem 2: monkey-test2.pddl

```
(define (problem monkey-test2)
  (:domain monkey-domain)
  (:objects p1 p2 p3 p4 p6)
  (:init (location p1) (location p2)
         (location p3) (location p4) (location p6)
         (at monkey p1) (on-floor)
         (at box p2) (at bananas p3) (at knife p4)
         (at waterfountain p3) (at glass p6))
  (:goal (and (hasbananas) (haswater))))
```

**go-to p4 p1**
**get-knife p4**
**go-to p6 p4**
**pick-glass p6**

**go-to p2 p6**
**push-box p3 p2**
**climb p3**
**get-water p3**
**grab-bananas p3**

**time = 70 msec.**

# Planning representation

**Suppose that the monkey wants to fool the scientists, who are off to tea, by grabbing the bananas, but leaving the box in its original place. Can this goal be solved by a STRIPS-style system?**

```
;
; Modelling the Wumpus World in PDDL: 1st try...
; by: Patrik Haslum
; Source web page:
;   http://users.cecs.anu.edu.au/~patrik/pddlman/wumpus.html
;

(define (domain wumpus-a)
  (:requirements :strips) ;; maybe not necessary

  (:predicates
   (adj ?square-1 ?square-2)
   (pit ?square)
   (at ?what ?square)
   (have ?who ?what)
   (dead ?who))

  (:action move
    :parameters (?who ?from ?to)
    :precondition (and (adj ?from ?to)
                       (not (pit ?to))
                       (at ?who ?from))
    :effect (and (not (at ?who ?from))
                 (at ?who ?to))
    )

  (:action take
    :parameters (?who ?what ?where)
    :precondition (and (at ?who ?where)
                       (at ?what ?where))
    :effect (and (have ?who ?what)
                 (not (at ?what ?where)))
    )

  (:action shoot
    :parameters (?who ?where ?arrow ?victim ?where-victim)
    :precondition (and (have ?who ?arrow)
                       (at ?who ?where)
                       (at ?victim ?where-victim)
                       (adj ?where ?where-victim))
    :effect (and (dead ?victim)
                 (not (at ?victim ?where-victim))
                 (not (have ?who ?arrow)))
    )
)
```

```
(define (problem wumpus-a-1)
  (:domain wumpus-a)
  (:objects
   sq-1-1 sq-1-2 sq-1-3
   sq-2-1 sq-2-2 sq-2-3
   the-gold
   the-arrow
   agent
   wumpus)

  (:init
   (adj sq-1-1 sq-1-2) (adj sq-1-2 sq-1-1)
   (adj sq-1-2 sq-1-3) (adj sq-1-3 sq-1-2)
   (adj sq-2-1 sq-2-2) (adj sq-2-2 sq-2-1)
   (adj sq-2-2 sq-2-3) (adj sq-2-3 sq-2-2)
   (adj sq-1-1 sq-2-1) (adj sq-2-1 sq-1-1)
   (adj sq-1-2 sq-2-2) (adj sq-2-2 sq-1-2)
   (adj sq-1-3 sq-2-3) (adj sq-2-3 sq-1-3)

   (pit sq-1-2)

   (at the-gold sq-1-3)
   (at agent sq-1-1)
   (have agent the-arrow)
   (at wumpus sq-2-3))

  (:goal (and (have agent the-gold) (at agent sq-1-1)))
  )

Resulting plan:

(MOVE THE-GOLD SQ-1-3 SQ-2-3)
(MOVE THE-GOLD SQ-2-3 SQ-2-2)
(MOVE THE-GOLD SQ-2-2 SQ-2-1)
(MOVE THE-GOLD SQ-2-1 SQ-1-1)
(TAKE AGENT THE-GOLD SQ-1-1)
```

```
;
; Modelling the Wumpus World in PDDL: 2nd try...
; by: Patrik Haslum
; Source web page:
;   http://users.cecs.anu.edu.au/~patrik/pddlman/wumpus.html
;

(define (domain wumpus-b)
  (:requirements :strips)
  (:predicates
   (adj ?square-1 ?square-2)
   (pit ?square)

   (at ?what ?square)
   (have ?who ?what)

   (takeable ?what)
   (is-gold ?what)
   (is-arrow ?what)

   (alive ?who)
   (dead ?who))

  (:action move
    :parameters (?who ?from ?to)
    :precondition (and (alive ?who)
                       (at ?who ?from)
                       (adj ?from ?to)
                       (not (pit ?to)))
    :effect (and (not (at ?who ?from))
                 (at ?who ?to))
    )

  (:action take
    :parameters (?who ?what ?where)
    :precondition (and (alive ?who)
                       (takeable ?what)
                       (at ?who ?where)
                       (at ?what ?where))
    :effect (and (have ?who ?what)
                 (not (at ?what ?where)))
    )

  (:action shoot
    :parameters (?who ?where ?arrow ?victim ?where-victim)
    :precondition (and (alive ?who)
                       (have ?who ?arrow)
                       (is-arrow ?arrow)
                       (at ?who ?where)
                       (alive ?victim)
                       (at ?victim ?where-victim)
                       (adj ?where ?where-victim))
    :effect (and (dead ?victim)
                 (not (alive ?victim))
                 (not (at ?victim ?where-victim))
                 (not (have ?who ?arrow)))
    )
)
```

```
(define (problem wumpus-b-1)
  (:domain wumpus-b)
  (:objects sq-1-1 sq-1-2 sq-1-3
            sq-2-1 sq-2-2 sq-2-3
            the-gold the-arrow
            agent wumpus)
  (:init (adj sq-1-1 sq-1-2) (adj sq-1-2 sq-1-1)
         (adj sq-1-2 sq-1-3) (adj sq-1-3 sq-1-2)
         (adj sq-2-1 sq-2-2) (adj sq-2-2 sq-2-1)
         (adj sq-2-2 sq-2-3) (adj sq-2-3 sq-2-2)
         (adj sq-1-1 sq-2-1) (adj sq-2-1 sq-1-1)
         (adj sq-1-2 sq-2-2) (adj sq-2-2 sq-1-2)
         (adj sq-1-3 sq-2-3) (adj sq-2-3 sq-1-3)
         (pit sq-1-2)
         (at the-gold sq-1-3)
         (is-gold the-gold)
         (takeable the-gold)
         (at agent sq-1-1)
         (alive agent)
         (have agent the-arrow)
         (is-arrow the-arrow)
         (takeable the-arrow)
         (at wumpus sq-2-3)
         (alive wumpus))
  (:goal (and (have agent the-gold)
              (at agent sq-1-1)
          ))
  )
```

Resulting plan:

```
(MOVE AGENT SQ-1-1 SQ-2-1)
(MOVE AGENT SQ-2-1 SQ-2-2)
(MOVE AGENT SQ-2-2 SQ-2-3)
(MOVE AGENT SQ-2-3 SQ-1-3)
(TAKE AGENT THE-GOLD SQ-1-3)
(MOVE AGENT SQ-1-3 SQ-2-3)
(MOVE AGENT SQ-2-3 SQ-2-2)
(MOVE AGENT SQ-2-2 SQ-2-1)
(MOVE AGENT SQ-2-1 SQ-1-1)
```

```
;
; Modelling the Wumpus World in PDDL: 3rd time’s a charm...
; by: Patrik Haslum
; Source web page:
;   http://users.cecs.anu.edu.au/~patrik/pddlman/wumpus.html
;

(define (domain wumpus-c)
  (:requirements :strips)
  (:predicates
   (at ?what ?square)
   (adj ?square-1 ?square-2)
   (pit ?square)
   (wumpus-in ?square)
     ;; <-> (exists ?x (and (is-wumpus ?x) (at ?x ?square) (not (dead ?x))
   (have ?who ?what)
   (is-agent ?who)
   (is-wumpus ?who)
   (is-gold ?what)
   (is-arrow ?what)
   (dead ?who))

  (:action move-agent
    :parameters (?who ?from ?to)
    :precondition (and (is-agent ?who)
                       (at ?who ?from)
                       (adj ?from ?to)
                       (not (pit ?to))
                       (not (wumpus-in ?to)))
    :effect (and (not (at ?who ?from))
                 (at ?who ?to))
    )

  (:action take
    :parameters (?who ?what ?where)
    :precondition (and (is-agent ?who)
                       (at ?who ?where)
                       (at ?what ?where))
    :effect (and (have ?who ?what)
                 (not (at ?what ?where)))
    )

  (:action shoot
    :parameters (?who ?where ?with-what ?victim ?where-victim)
    :precondition (and (is-agent ?who)
                       (have ?who ?with-what)
                       (is-arrow ?with-what)
                       (at ?who ?where)
                       (is-wumpus ?victim)
                       (at ?victim ?where-victim)
                       (adj ?where ?where-victim))
    :effect (and (dead ?victim)
                 (not (wumpus-in ?where-victim))
                 (not (have ?who ?with-what)))
    )
```

```
  (:action move-wumpus
    :parameters (?who ?from ?to)
    :precondition (and (is-wumpus ?who)
                       (at ?who ?from)
                       (adj ?from ?to)
                       (not (pit ?to))
                       (not (wumpus-in ?to)))
    :effect (and (not (at ?who ?from))
                 (at ?who ?to)
                 (not (wumpus-in ?from))
                 (wumpus-in ?to))
    )

)

(define (problem wumpus-c-1)
  (:domain wumpus-c)
  (:objects sq-1-1 sq-1-2 sq-1-3
            sq-2-1 sq-2-2 sq-2-3
            the-gold the-arrow
            agent wumpus)
  (:init (adj sq-1-1 sq-1-2) (adj sq-1-2 sq-1-1)
         (adj sq-1-2 sq-1-3) (adj sq-1-3 sq-1-2)
         (adj sq-2-1 sq-2-2) (adj sq-2-2 sq-2-1)
         (adj sq-2-2 sq-2-3) (adj sq-2-3 sq-2-2)
         (adj sq-1-1 sq-2-1) (adj sq-2-1 sq-1-1)
         (adj sq-1-2 sq-2-2) (adj sq-2-2 sq-1-2)
         (adj sq-1-3 sq-2-3) (adj sq-2-3 sq-1-3)
         (pit sq-1-2)
         (is-gold the-gold)
         (at the-gold sq-1-3)
         (is-agent agent)
         (at agent sq-1-1)
         (is-arrow the-arrow)
         (have agent the-arrow)
         (is-wumpus wumpus)
         (at wumpus sq-2-3)
         (wumpus-in sq-2-3))
  (:goal (and (have agent the-gold) (at agent sq-1-1)))
  )

Resulting plan:

(MOVE-AGENT AGENT SQ-1-1 SQ-2-1)
(MOVE-AGENT AGENT SQ-2-1 SQ-2-2)
(SHOOT AGENT SQ-2-2 THE-ARROW WUMPUS SQ-2-3)
(MOVE-AGENT AGENT SQ-2-2 SQ-2-3)
(MOVE-AGENT AGENT SQ-2-3 SQ-1-3)
(TAKE AGENT THE-GOLD SQ-1-3)
(MOVE-AGENT AGENT SQ-1-3 SQ-2-3)
(MOVE-AGENT AGENT SQ-2-3 SQ-2-2)
(MOVE-AGENT AGENT SQ-2-2 SQ-2-1)
(MOVE-AGENT AGENT SQ-2-1 SQ-1-1)
```

```
;
; Modelling the Wumpus World in PDDL: using ADL...
; by: Patrik Haslum
; Source web page:
;   http://users.cecs.anu.edu.au/~patrik/pddlman/wumpus.html
;

(define (domain wumpus-adl)
  (:requirements :adl :typing)

  ;; object types
  (:types agent wumpus gold arrow square)

  (:predicates
   (adj ?square-1 ?square-2 - square)
   (pit ?square - square)
   (at ?what ?square)
   (have ?who ?what)
   (alive ?who))

  (:action move
    :parameters (?who - agent ?from - square ?to - square)
    :precondition (and (alive ?who)
                       (at ?who ?from)
                       (adj ?from ?to)
                        )
    :effect (and (not (at ?who ?from))
                 (at ?who ?to)

                 (when (pit ?to)
                   (and (not (alive ?who))))

                 (when (exists (?w - wumpus) (and (at ?w ?to) (alive ?w)))
                   (and (not (alive ?who)))))
    )

  (:action take
    :parameters (?who - agent ?where - square ?what)
    :precondition (and (alive ?who)
                       (at ?who ?where)
                       (at ?what ?where))
    :effect (and (have ?who ?what)
                 (not (at ?what ?where)))
    )

  (:action shoot
    :parameters (?who - agent ?where - square ?with-arrow - arrow
                 ?victim - wumpus ?where-victim - square)
    :precondition (and (alive ?who)
                       (have ?who ?with-arrow)
                       (at ?who ?where)
                       (alive ?victim)
                       (at ?victim ?where-victim)
                       (adj ?where ?where-victim))
    :effect (and (not (alive ?victim))
                 (not (have ?who ?with-arrow)))
    )
)
```

```
(define (problem wumpus-adl-1)
  (:domain wumpus-adl)

  (:objects
   sq-1-1 sq-1-2 sq-1-3 sq-2-1 sq-2-2 sq-2-3 - square
   the-gold - gold
   the-arrow - arrow
   agent-1 - agent
   wumpus-1 - wumpus)

  (:init (adj sq-1-1 sq-1-2) (adj sq-1-2 sq-1-1)
         (adj sq-1-2 sq-1-3) (adj sq-1-3 sq-1-2)
         (adj sq-2-1 sq-2-2) (adj sq-2-2 sq-2-1)
         (adj sq-2-2 sq-2-3) (adj sq-2-3 sq-2-2)
         (adj sq-1-1 sq-2-1) (adj sq-2-1 sq-1-1)
         (adj sq-1-2 sq-2-2) (adj sq-2-2 sq-1-2)
         (adj sq-1-3 sq-2-3) (adj sq-2-3 sq-1-3)
         (pit sq-1-2)
         (at the-gold sq-1-3)
         (at agent-1 sq-1-1)
         (alive agent-1)
         (have agent-1 the-arrow)
         (at wumpus-1 sq-2-3)
         (alive wumpus-1))

  (:goal (and (have agent-1 the-gold) (at agent-1 sq-1-1) (alive agent-1)))
  )
```